O'REILLY®

Databricks Certified Generative AI Engineer Associate Study Guide

Generative AI with Databricks



Early Release

RAW & UNEDITED

Rajaniesh Kaushikk

Databricks Certified Generative AI Engineer Associate Study Guide

Generative AI with Databricks

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Rajaniesh Kaushikk



Databricks Certified Generative AI Engineer Associate Study Guide

by Rajaniesh Kaushikk

Copyright © 2026 Rajaniesh Kaushikk. All rights reserved.

Published by O’Reilly Media, Inc. , 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (https://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com* .

Acquisitions Editor: Andy Kwan

Development Editor: Corbin Collins

Production Editor: Jonathon Owen

Cover Designer: Karen Montgomery

Cover Illustrator: FILL IN

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

May 2026: First Edition

Revision History for the Early Release

- 2025-08-14: First Release

See https://oreilly.com/catalog/errata.csp?isbn=9798341623453 for release details.

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. Databricks Certified Generative AI Engineer Associate Study Guide, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

The views expressed in this work are those of the author(s) and do not represent the publisher’s views. While the publisher and the author(s) have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author(s) disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-62340-8

[FILL IN]

Brief Table of Contents (*Not Yet Final*)

Chapter 1: Exam Details and Resources (available)

Chapter 2: Designing Generative AI Applications (available)

Chapter 3: Preparing and Chunking Data for RAG Applications (available)

Chapter 4: Building GenAI Applications with Python and LangChain (unavailable)

Chapter 5 Deploying and Integrating RAG Systems on Databricks (unavailable)

Chapter 6 Managing Models with MLflow and Unity Catalog (unavailable)

Chapter 7 Responsible AI: Guardrails and Governance in GenAI (unavailable)

8 Monitoring and Evaluating LLMs in Production Environments (unavailable)

9 Scaling AI Solutions with Vector Search and Mosaic (unavailable)

Chapter 10 Databricks GenAI Associate Certification Preparation (unavailable)

Chapter 1. Exam Details and Resources

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at *ccollins@oreilly.com*.

This chapter comprehensively overviews the Databricks Certified Generative AI Engineer Associate certification exam. It outlines the exam structure, objectives, and recommended study resources to help you prepare effectively. Knowing the certification details, you can proceed directly to the subsequent chapters covering technical concepts and hands-on exercises.

The Databricks Certified Generative AI Engineer Associate certification validates an individual's ability to design and implement LLM-enabled solutions using Databricks. It assesses skills in problem decomposition, model selection, application development, and Databricks-specific tools such as Vector Search, Model Serving, MLflow, and Unity Catalog.

This certification is ideal for AI engineers, data scientists, and solution architects who want to develop, deploy, and govern performant Retrieval-Augmented Generation (RAG) applications and LLM chains using Databricks.

Exam Objectives

The exam is structured into six key domains that assess both conceptual knowledge and hands-on experience:

1. Design Applications (14%)
    - Formulating effective prompts for structured outputs
    - Selecting appropriate models and chain components
    - Translating business use cases into AI pipeline descriptions
    - Defining tools for multi-stage reasoning
2. Data Preparation (14%)
    - Implementing chunking strategies for large documents
    - Filtering extraneous content to improve RAG quality
    - Extracting document content using Python libraries
    - Storing and retrieving structured data from the Unity Catalog
3. Application Development (30%)
    - Selecting retrieval tools and embedding models
    - Implementing LLM guardrails to prevent hallucinations
    - Constructing effective, prompt templates for AI agents
    - Evaluating and optimizing AI application performance
4. Assembling and Deploying Applications (22%)
    - Coding LLM pipelines using LangChain and PyFunc models

– Deploying models with Databricks Model Serving

– Implementing Vector Search for semantic search

– Registering models in Unity Catalog using MLflow

5. Governance (8%)

– Applying data masking techniques for privacy

– Implementing security guardrails against adversarial inputs

– Ensuring compliance with legal and licensing requirements

6. Evaluation and Monitoring (12%)

– Choosing the best LLM architecture based on evaluation metrics

– Implementing MLflow tracking for model performance

– Monitoring cost and inference logs in Databricks

Setting Up Your Databricks Workspace

To get hands-on experience with LLM model training, Vector Search, and Model Serving, you need a Databricks workspace. Follow these steps to set up your environment:

Step 1: Sign up for Databricks

1. Visit Databricks and sign up for an account.

2. Choose Cloud Provider Option (AWS, Azure, GCP).

3. Complete the registration and verify your email.

Step 2: Create a Databricks Workspace

1. Log in to the Databricks Console and navigate to Workspaces.
2. Click Create Workspace and select your cloud provider.
3. Choose a region closest to your location for lower latency.
4. Configure the Workspace with default settings and click Deploy. Please make sure to use the Premium edition for workspace selection.
5. Configure Machine Learning Runtime Clusters. You can select any suitable runtime.

Step 4: Create a Notebook and test your setup

1. Navigate to Workspace > Users > Create Notebook.
2. Name the Notebook (for example, Generative_AI_Exam_Practice).
3. Select Python as the default language.
4. Run the following test command to verify the installation:

```
import mlflow
print(mlflow.__version__)
```

> **TIP**
>
> Ensure your Workspace is configured before attempting exam-related exercises.

Exam Format and Outline

Table 1-1 summarizes the exam format. It outlines key logistical and structural aspects of the certification exam, such as duration, number of questions, language support, and delivery platform. This table is a quick reference for candidates preparing to schedule and attempt the exam.

Table 1-1. Exam format

Attribute	Details
Number of Questions	45 Multiple-choice / multiple-selection
Time Limit	90 minutes
Registration Fee	$200 USD
Delivery Method	Online Proctored at https://webassessor.com/databricks
Allowed Test Aids	None
Exam Languages	English, 日本語, Português BR, 한국어
Prerequisites	None, but 6+ months of hands-on experience is recommended
Validity Period	2 years

The exam is divided into six key sections, each assessing critical skills in generative AI development on Databricks. Focus on the sections with the highest weight, such as Application Development (30%) and Assembling and Deploying Applications (22%), to maximize your score.

Section 1: Design Applications

Designing effective prompts and selecting the right models are foundational skills for building high-quality LLM applications. Ensure you understand how different LLM architectures process inputs and generate responses:

- Design a prompt that elicits a specifically formatted response
- Select model tasks to accomplish a given business requirement
- Select chain components for a desired model input and output
- Translate business use case goals into a description of the desired inputs and outputs for the AI pipeline
- Define and order tools that gather knowledge or take actions for multi-stage reasoning

Section 2: Data Preparation

Poor data preparation can lead to degraded AI performance, inaccurate responses, and increased inference costs. Be sure to optimize your chunking strategies and document filtering methods:

- Apply a chunking strategy for a given document structure and model constraints
- Filter extraneous content in source documents that degrades the quality of an RAG application
- Choose the appropriate Python package to extract document content from the provided source data and format
- Define operations and sequence to write the given chunked text into Delta Lake tables in the Unity Catalog
- Identify needed source documents that provide the necessary knowledge and quality for a given RAG application
- Identify prompt/response pairs that align with a given model task
- Use tools and metrics to evaluate retrieval performance

Section 3: Application Development

Databricks provides several tools to help streamline generative AI development. Understanding how to leverage these tools effectively will be key to passing the exam:

LangChain

> Useful for building RAG pipelines and connecting external data sources.

MLflow

> Essential for tracking experiments, hyperparameter tuning, and model evaluation.

Vector Search

> Enhances semantic search and retrieval-based augmentation.

Experiment with these tools inside Databricks Notebooks to build hands-on expertise before taking the exam:

- Create tools needed to extract data for a given data retrieval need.
- Select LangChain/similar tools for use in a generative AI application.
- Identify how prompt formats can change model outputs and results.
- Qualitatively assess responses to identify common issues such as quality and safety.
- Select a chunking strategy based on model and retrieval evaluation.
- Augment a prompt with additional context from a user’s input based on key fields, terms, and intents.

- Create a prompt that adjusts an LLM’s response from a baseline to a desired output.
- Implement LLM guardrails to prevent negative outcomes.
- Write metaprompts that minimize hallucinations or leaking private data.
- Build agent prompt templates exposing available functions.
- Select the best LLM based on the attributes of the application to be developed.
- Select an embedding model context length based on source documents, expected queries, and optimization strategy.
- Select a model from a model hub or marketplace for a task based on model metadata/model cards.
- Select the best model for a given task based on common metrics generated in experiments.

Section 4: Assembling and Deploying Applications

Learn the core steps to transition your generative AI application from development to production in Databricks:

- Registering models in Unity Catalog for governance.
- Deploying MLflow models with pre- and post-processing.
- Setting up Vector Search for real-time semantic retrieval.

Remember that deployment in Databricks is different from traditional ML workflows. Understand how Databricks Model Serving works and how to optimize endpoints for scalability:

- Code a chain using a pyfunc model with pre- and post-processing.
- Control access to resources from model serving endpoints.
- Code a simple chain according to requirements.
- Code a simple chain using LangChain.
- Choose the basic elements needed to create a RAG application: model flavor, embedding model, retriever, dependencies, input examples, and model signature.
- Register the model to Unity Catalog using MLflow.
- Sequence the steps needed to deploy an endpoint for a basic RAG application.
- Create and query a Vector Search index.
- Identify how to serve an LLM application that leverages Foundation Model APIs.
- Identify resources needed to serve features for a RAG application.

Section 5: Governance

Governance is crucial in safeguarding your GenAI solutions. Learn the key concepts around Governance:

- Use masking techniques as guard rails to meet a performance objective.
- Select guardrail techniques to protect against malicious user inputs to a genAI application.
- Recommend an alternative for problematic text mitigation in a data source feeding a RAG application.
- Use legal/licensing requirements for data sources to avoid legal risk.

Section 6: Evaluation and Monitoring

Use inference logging to track real-time model performance in production. This will help identify errors, reduce costs, and improve response accuracy:

- Select an LLM choice (size and architecture) based on quantitative evaluation metrics.
- Select key metrics to monitor for a specific LLM deployment scenario.
- Evaluate model performance in a RAG application using MLflow.
- Use inference logging to assess deployed RAG application performance.
- Use Databricks features to control LLM costs for RAG applications.

Key Preparation Strategies

Success in the Databricks Certified Generative AI Engineer Associate exam requires a structured study plan, hands-on experience, and familiarity with Databricks' AI tools. This section discusses some key strategies to help you prepare effectively:

Understanding the Exam Blueprint

Before diving into exam prep, it's important to familiarize yourself with the structure and scope of the exam. The blueprint outlines the domains you'll be tested on and the weight of each section, guiding how you should prioritize your study time.

- Review the exam objectives and section weightage to focus on high-impact topics.
- Pay extra attention to Application Development (30%) and Assembling and Deploying Applications (22%) as they carry the

most weight.

- Study the Databricks exam guide and familiarize yourself with exam expectations.

Gaining Hands-On Experience

Before you can succeed on this exam, doing more than just reading is essential. Gaining hands-on experience is the best way to internalize concepts and workflows. Practical exposure allows you to troubleshoot real issues, understand tool behavior, and build the confidence to tackle real-world GenAI use cases within Databricks. This section outlines the key areas to focus on during your hands-on practice.

- Create and configure a Databricks workspace to practice exam-relevant tasks.
- Work on real-world use cases by building RAG applications, implementing Vector Search, and managing models with MLflow.
- Deploy LLM-powered applications and experiment with Unity Catalog for governance.

Leveraging Official Databricks Documentation

Review Databricks documentation and AI whitepapers for deeper insights.

Practicing with Sample Questions and Mock Exams

Practicing with sample questions is one of the most effective ways to familiarize yourself with the exam's structure, difficulty level, and style of questioning. It helps reinforce your conceptual understanding and improves your speed and accuracy under time constraints.

- Work through Databricks practice exams and sample questions.
- Simulate real exam conditions by setting a 90-minute timer and solving multiple-choice and multi-selection questions.

- Focus on troubleshooting common issues and optimizing AI workflows.

Mastering Prompt Engineering and Model Selection

Mastering prompt engineering and model selection is at the heart of building effective generative AI applications. These skills enable you to generate accurate and relevant outputs and ensure your models operate efficiently, safely, and at scale within Databricks. Understanding how to frame questions, inject contextual knowledge, and select the right model for the task directly impacts both performance and user experience.

- Understand how different prompting techniques (zero-shot, few-shot, chain-of-thought) impact model performance.
- Learn to select the right model for different tasks based on accuracy, latency, and cost considerations.
- Optimize model responses by applying guardrails, metaprompts, and fine-tuning methods.

Practical Exam Tips

This section collects actionable tips to improve exam performance and avoid common pitfalls.

Before the Exam:

Ensure your Databricks Workspace is ready

> Set up clusters, Unity Catalog, MLflow, and Model Serving.

Review the Official Exam Guide

> Ensure you study the latest exam version and objectives.

Bookmark key documentation pages

Use Databricks docs for quick reference (if permitted in the exam).

During the Exam:

Manage your time effectively

Spend no more than 2 minutes per question and flag difficult ones for later review.

Read questions carefully

Identify key requirements and eliminate incorrect answers systematically.

Use logical troubleshooting approaches

For scenario-based questions, apply debugging best practices.

- Staying hydrated and full is important.

Common mistakes to avoid:

Not practicing in Databricks

The exam tests practical implementation, so theoretical knowledge alone is insufficient.

Ignoring AI governance concepts

Be prepared for questions about data security, model monitoring, and compliance.

Skipping experiment tracking

Mlflow is crucial for model versioning, performance tracking, and deployment.

Certification Benefits and Career Impact

Why should you get certified? Earning the Databricks Certified Generative AI Engineer Associate credential validates your expertise in building LLM-powered applications using Databricks' AI ecosystem. Here's why this certification is valuable:

Industry recognition

> Demonstrates proficiency in LLM application design, data preparation, and deployment.

Career growth

> Opens AI engineering, data science, and MLOps role**s**.

Higher earning potential

> Certified professionals often command higher salaries due to specialized expertise.

Competitive advantage

> Sets you apart in a job market increasingly focused on AI-driven solutions.

Networking opportunities

> Access Databricks certified professional community and career resources.

What can the certification help you do? With this certification, you'll be able to apply LLMs and AI pipelines in multiple industries:

Healthcare

> Ai-powered medical chatbots, patient data retrieval, and disease prediction models.

Finance

Fraud detection, algorithmic trading insights, and automated customer support.

Retail and e-commerce

Personalized recommendations, AI-powered search, and virtual assistants.

Enterprise AI

Automating customer inquiries, summarizing corporate documents, and enhancing productivity with AI-driven workflows.

> **TIP**
>
> Highlighting this certification on LinkedIn, your resume, or during job interviews can help showcase your expertise in generative AI engineering.

Summary

This brief first chapter examined the Databricks Certified Generative AI Engineer Associate exam, covering its format, objectives, and essential preparation strategies. Understanding the six key domains—from designing applications and data preparation to governance and evaluation—is crucial for success.

Leveraging official Databricks training, including instructor-led and self-paced courses, will ensure a solid foundation in generative AI engineering. Additionally, gaining hands-on experience through Databricks Notebooks, MLflow tracking, Vector Search, and Model Serving will significantly enhance your ability to solve real-world AI challenges.

Use practice exams, sample questions, and official Databricks documentation to reinforce your knowledge. These resources will help identify knowledge gaps and improve confidence in tackling exam

scenarios. Mastering prompt engineering, model selection, application development, and deployment best practices will prepare you to successfully implement AI-powered solutions within Databricks.

The next chapter will explore LLM application design and prompt engineering, which are critical to building effective and reliable AI-driven applications using Databricks' robust ecosystem.

Chapter 2. Designing Generative AI Applications

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at *ccollins@oreilly.com*.

Generative AI applications only succeed when designed intentionally, with a deep understanding of how different components interact within a system. Designing a successful AI application goes far beyond feeding a prompt to a model; it involves mapping the business problem to the right model capability, shaping the prompt to reflect operational needs, and coordinating a series of reasoning steps that work together reliably. Without these elements, generative models can produce inconsistent, unstructured, or even misleading outputs that organizations cannot integrate into enterprise workflows.

In this chapter, you'll go beyond basic prompting. You'll learn how to deconstruct a business use case into logical tasks, identify the most appropriate LLM behavior for each stage, and implement modular prompt chains that can be validated, reused, and scaled. You'll explore how to match model capabilities to structured outputs, use reasoning chains to orchestrate complex decisions, and evaluate your design using practical

techniques. These skills aren't just theoretical—they align directly with the Databricks Certified Generative AI Associate exam, which assesses your ability to reason through system design, workflow structure, and performance trade-offs in AI-driven applications.

By mastering this design process, you will gain the ability to deliver AI solutions that are not only intelligent but also robust, explainable, and business-ready.

Prerequisites

Before starting this chapter, make sure you're familiar with:

- What large language models (LLMs) are and how they work at a high level
- The concept of model inference and common output types (e.g., text, structured data)
- Basic Python syntax and Databricks notebooks

Learning Objectives

By the end of this chapter, you will be able to:

- Analyze business problems and match them with appropriate LLM task types (e.g., classification, extraction, transformation)
- Design structured and modular prompts that elicit consistent, machine-readable outputs
- Use `PromptTemplate` and LLMChain to build and reuse prompts programmatically
- Assemble multi-step workflows using LangChain chains, tools, and agents in Databricks

- Evaluate pipeline effectiveness using prompt-task alignment, tool ordering, and reasoning trace outputs

Crafting Prompts for Structured Output

Large language models (LLMs) offer flexibility, but without precise instructions, they often generate inconsistent, verbose, or unpredictable responses. For many enterprise applications—such as legal audits, financial summaries, or customer support workflows—you need model outputs that follow a specific structure, such as JSON, SQL, or tabular formats. Designing prompts that constrain and direct the model is essential for downstream integration and automation.

Let's start with a telecom example. Imagine you're building a support chatbot and input the following prompt:

```
Summarize this support ticket
```

The model may return a paragraph of unstructured text. Instead, you could specify the following:

```
Summarize the following customer support ticket and return the result
in JSON format, including the fields customer_id, issue_summary, and
urgency_level.
```

Now you are guiding the model to output machine-readable, predictable content. That precision reduces ambiguity and makes integration with downstream systems, such as ticket prioritization engines or dashboards, much easier.

Real-World Examples of Structured Prompting

Structured prompting becomes critical when business requirements demand consistent, parseable, and actionable outputs from language models. Rather than leave the model to decide the format or structure, you define the shape of the response to match what your downstream systems or human users

expect. This is especially important in regulated industries or high-stakes automation pipelines where unpredictability is a liability.

Here are some business cases where structured prompting ensures reliable performance:

Legal compliance audits

> A legal firm wants to identify contract clauses that mention termination conditions and store them in a compliance report. Instead of vague or interpretive summaries, the goal is to extract and precisely tag each relevant clause. The prompt might be as follows:
>
> ```
> Extract any clause mentioning 'termination' from this contract and
> return a JSON object with fields clause_id, text_excerpt, and
> risk_flag.
> ```
>
> This approach enables the automated flagging of risk-prone clauses, speeding up the document review process.

Finance report summarization

> An analyst requests financial insights from quarterly earnings PDFs. Rather than read lengthy prose or manually pull metrics, the analyst wants structured data suitable for entry into an internal dashboard. Here is a sample prompt:
>
> ```
> Summarize this Q3 earnings report in JSON format with keys:
> revenue, net_income, guidance, and notable_changes.
> ```
>
> This method standardizes the analysis of financial documents across departments.

Healthcare pre-visit intake

For a telehealth intake form, clinicians need to extract symptoms, duration, and medical history efficiently. The LLM supports automated triage by structuring this information:

```
From this patient note, extract symptom_list, duration_days, and
preexisting_conditions as JSON.
```

Structured data entry helps standardize diagnostic workflows and enhances interoperability with electronic health record (EHR) systems.

Retail inventory queries

A product assistant tool receives natural language questions, such as this:

```
Show me red dresses under $100.
```

Rather than generate a paragraph, the LLM should extract structured query fields such as category, color, and price ceiling:

```
Extract product_type, color, and max_price from this query and
return as a JSON object.
```

This technique ensures a clean query to the product catalog API.

Insurance claim validation

Agents may input long, detailed descriptions of incidents. Rather than summarize, the system needs to extract relevant fields such as `claim_type`, `severity`, and `fraud_flag`:

```
Extract fields from this claim: claim_type, issue_description,
risk_score, fraud_flag. Return as JSON.
```

This method accelerates downstream fraud analysis and case routing.

These use cases benefit from a consistent structure, enabling analytics pipelines, dashboards, or case management systems to utilize the model's output without requiring human review.

Strategies for Structured Prompting

Crafting structured prompts is not just about getting the model to respond—it's about guiding it to produce output that's consistent, validated, and immediately useful in a production context. Whether you are integrating with downstream systems, building user-facing features, or automating decision-making, your prompt must impose the right structure without sacrificing clarity. This section presents two core strategies you can use to steer the model toward consistent formats reliably: using delimiters and schemas and including few-shot examples.

Using delimiters and schemas

Delimiters are clear markers that separate instructions from input, helping the model distinguish between what to process and what to follow. *Schemas* define the structure of the desired output. For example, wrap your input in `<input>...</input>` and explicitly specify your output format:

`{document_text}` returns a JSON object with keys: topic, summary, sentiment.

This technique works particularly well in applications such as survey analysis or feedback processing, where you need to extract standard fields from varied inputs.

Including few-shot examples

Few-shot prompting involves providing 1–2 examples within the prompt to illustrate the expected behavior. These examples serve as training hints, showing the model how to behave without requiring actual fine-tuning. Here's a typical example:

Input: `Customer reports poor audio quality during calls.`

Expected output (JSON):

```
 { "category": "technical", "sentiment": "negative", "urgency":
"high" }
```

Then continue: `Now extract the same fields from the following input: {customer_feedback}`

This technique is especially useful when strict formatting is required.

> **TIP**
>
> Use double curly braces like *{{placeholder}}* when building prompt templates. This enables easy substitution using orchestration tools or Python string injection.

Table 2-1 illustrates the differences between vague prompts and schema-driven prompts in terms of reliability and usability.

Table 2-1. Comparison of vague vs. structured prompts

Use Case	Vague Prompt Example	Structured Prompt Example
Customer support	`Summarize this ticket.`	`Summarize this ticket in JSON format with the following fields: customer_id, issue_summary, and urgency_level.`
Legal compliance	`What does this clause say?`	`Extract any clause related to 'termination' and return: clause_id, text_excerpt.`
Financial reporting	`Tell me about the Q3 results.`	`Summarize Q3 results with keys: revenue, profit_margin, forecast, challenges.`

This structured approach reduces errors, improves consistency, and makes model responses easier to parse and validate. It's particularly important when model outputs feed into automated systems, where unstructured or inconsistent results can cause failures.

Using Prompt Templates in Code

You can implement reusable prompt templates in code using LangChain, which provides a structured way to design and manage prompts for repeated use. These templates act as blueprints that clearly define the instruction format and include placeholders for dynamic values, such as user inputs or variables pulled from a database. By centralizing the structure of your prompts, you ensure that each instance follows the same logic and formatting rules, reducing the likelihood of errors and inconsistencies.

For example, suppose you're building a clinical note summarization system. In that case, you might have a prompt template with a static instruction, such as *"Extract diagnosis codes"*, and a dynamic placeholder for the note text. Instead of rewriting the prompt every time, you programmatically

inject new note data into the placeholder, maintaining output structure across hundreds of cases.

This templated approach makes it easier to debug errors, standardize behavior across environments, and quickly scale prompt variations. It also integrates well with orchestration tools that dynamically generate prompts in response to user queries or workflow steps. Example 2-1 shows a structured prompt using LangChain.

Example 2-1. ICD-10 structured prompt using LangChain

```
from langchain.prompts import PromptTemplate
prompt = PromptTemplate.from_template(
    "Extract all ICD-10 codes from this note:\n{note_text}\nReturn them
in JSON under 'diagnosis_codes'."
)
```

This format allows you to dynamically plug in `note_text` while maintaining strict control over the output format. Using prompt templates in this way ensures consistency in outputs, supports rapid testing across different inputs, and facilitates easy integration of the model into larger LLM workflows and chains. It also helps centralize prompt logic, making it easier to maintain and update across multiple use cases.

Figure 2-1 illustrates how a prompt template distinguishes between static text, dynamic placeholders (e.g., `{{customer_issue}}`), and the expected output schema. The system injects each placeholder programmatically before sending the prompt to the large language model (LLM).

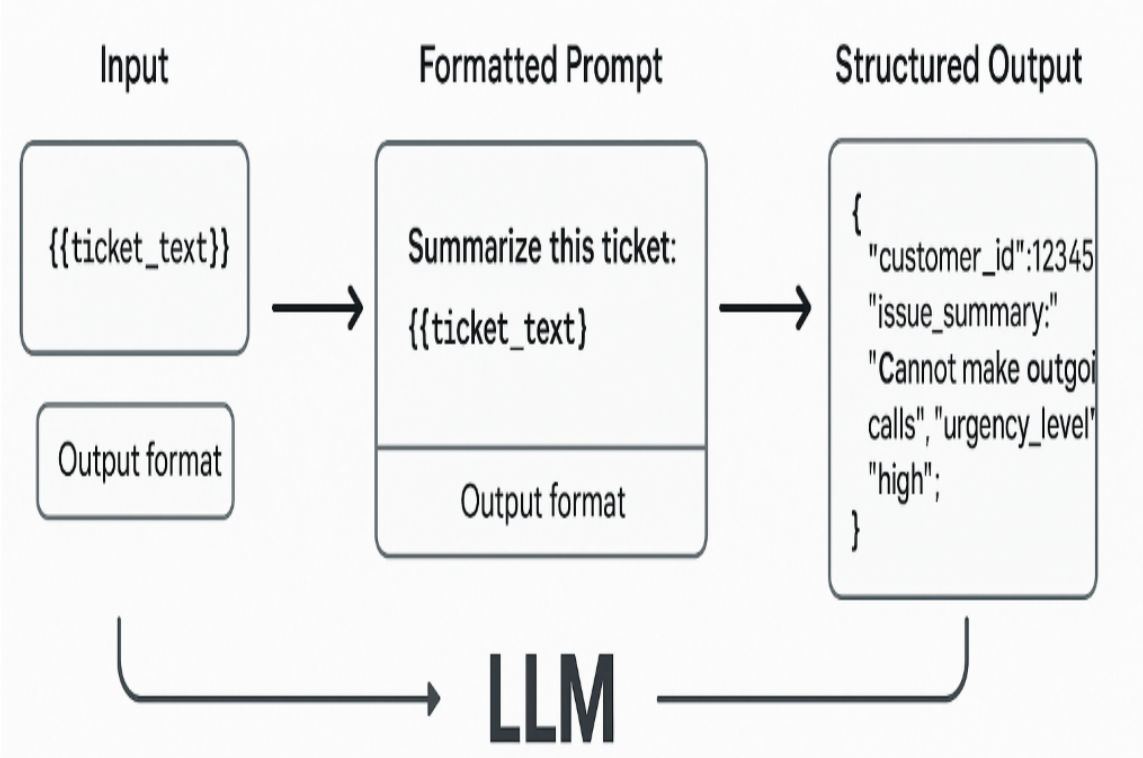


Figure 2-1. Prompt template structure diagram

Matching Model Tasks to Use Cases

Designing effective prompts begins with a clear understanding of what you're asking the language model to do—this is the *task type*. Whether you're summarizing, classifying, extracting, or transforming data, the model's behavior must align with the specific needs of your business case. Misalignment between task type and use case is one of the most common and costly sources of failure in generative AI deployments. For example, asking a model to *"summarize"* a medical report when what you need is a list of diagnosis codes results in verbose, unstructured outputs that downstream systems cannot use.

Every business scenario requires a different interaction style with the model. Some may require open-ended content generation, while others need precise data extraction. Selecting the correct task type allows you to write focused, goal-aligned prompts, apply the right evaluation criteria (e.g., accuracy, relevance, completeness), and anticipate the exact shape and structure of the model's output. This clarity not only improves performance

but also ensures smoother integration with dashboards, APIs, or workflow automation tools.

Common LLM task types

The most common task types in business-centric LLM applications include a diverse set of patterns that align with specific operational goals. Understanding when to use each of these enables developers and product teams to avoid vague or misleading outputs and to create workflows that are modular, robust, and automatable. Here are four foundational task types that frequently appear in enterprise systems:

Text generation

> Ideal for open-ended content creation, text generation enables the model to produce freeform text based on the given prompt. Use cases include writing product descriptions for e-commerce platforms, generating personalized emails in marketing campaigns, summarizing long reports, or drafting policy language in a consistent tone. For instance, a retail brand could use text generation to create dozens of unique yet brand-aligned product summaries from a central catalog.

Classification

> This task involves assigning a category, tag, or label to a given input. It's commonly applied in customer support systems to classify incoming emails by intent (e.g., *"cancel order"* or *"technical issue"*), in HR applications to categorize job applicants, or in content moderation workflows to flag sensitive material. Classification outputs are typically short, discrete values that rule engines or dashboards can consume.

Extraction

Use this task when your goal is to extract structured fields from messy or unstructured text. Extraction is common in various sectors, including healthcare (e.g., extracting ICD-10 codes or symptoms), finance (extracting transaction amounts or tax IDs from scanned invoices), and law (retrieving clause types from contracts). The output is typically a JSON object, a CSV row, or a dictionary that systems can parse programmatically.

Transformation

Transformation tasks are essential when you need to change the format, style, or purpose of a text. These tasks include translating a query into SQL, rephrasing legalese into plain English, converting user input into API requests, or transforming bullet points into professional emails. A sales CRM might use transformation to convert call notes into a summary suitable for executive reporting.

> **NOTE**
>
> Task selection guides everything from prompt design to evaluation. Start by clearly defining your desired output type—free text, list, JSON, category label—and work backward to identify the right model behavior.

We can look at task selection with the help of an example of a healthcare use case where a misaligned task type can produce undesired output.

Use case: Misaligned task type

Consider a healthcare analytics team that needs to extract diagnosis codes from unstructured doctor notes. A developer mistakenly prompts the model as if the goal were summarization:

```
Summarize the following clinical note.
```

The output might be a paragraph of medical narrative, which does not serve the team's needs. This mismatch between production and need is a classic misalignment—the team requires *extraction*, not *summarization*.

Here's a better prompt:

```
Extract all ICD-10 diagnosis codes from the following clinical note.
Return the result as a JSON list under the key diagnosis_codes.
```

This revised prompt aligns the task (extraction) with the goal (structured code retrieval), resulting in better outcomes. Misaligned prompts not only return incorrect results but also create confusion in downstream systems, which may rely on schema consistency for validation, filtering, or automation.

Contrasting use cases: Classification vs. extraction

To fully appreciate the distinction between classification and extraction, it's helpful to first ground these concepts in the context of a real-world workflow. Consider an enterprise HR department or legal team that processes large volumes of text data every day—from resumes to contracts to policy documents. These teams often face multiple questions: How do we sort this content for the right reviewer? How do we extract the key details without manual effort? Such scenarios are where choosing between classification and extraction becomes critical. We'll walk through two examples in detail: HR and legal operations.

Let's examine the differences between classification and extraction when applied in enterprise contexts by comparing their goals, outputs, and the prompts that activate each task.

Classification involves assigning a predefined label to a piece of input data, often to support routing, filtering, or analytics. *Extraction*, on the other hand, consists of extracting specific pieces of information from unstructured text to populate structured data systems. Understanding the contrast between these two task types helps ensure that LLM applications

are scoped correctly and deliver predictable, actionable results. Now let's understand it with two examples:

HR resume analytics

In the HR domain, companies receive a flood of resumes daily. The challenge is not only identifying the right candidates but doing so at scale, with consistency. LLMs can assist with both routing and data parsing depending on the specific prompt structure.

Classification

The goal here is to sort each resume into a predefined role category (e.g., software engineer, data analyst, product manager**).** This categorization is useful for automating job pipelines. Prompt:

```
Classify this candidate's resume as a software
engineer, data analyst, or product manager.
```

Output: A discrete label, like *software engineer*, suitable for filtering or queueing.

Extraction

Instead of just labeling, you may need to extract detailed fields from each resume for an applicant tracking system, such as years of experience, degrees, or technical skills. Prompt:

Extract `years_of_experience, education_level, and skills from this resume. Return as JSON.`

Output: Structured key-value pairs that support analytics, search filtering, or candidate ranking.

Legal contract analytics

Legal teams frequently review and analyze lengthy contracts that contain numerous clauses. Each clause may pertain to different legal functions, such as termination, liability, or payment terms. Prompting LLMs correctly enables both contract analysis and downstream compliance automation.

Classification

The first step might be labeling the paragraph or clause based on its legal type. This categorization enables the rapid identification of areas where to focus legal review. Prompt:

Classify `the following paragraph as liability, payment, or termination.`

Output: A category label, e.g., *termination*, to organize sections or prioritize legal reviews.

Extraction

After classification, you may need to extract specific obligations, such as a due date or monetary value. Prompt:

```
Extract the payment_amount and due_date from this contract

clause in JSON.
```

Output: `{ "payment_amount": "$5,000", "due_date": "2025-09-01" }` — structured data that feeds into a compliance tracker or risk dashboard.

Choosing the right model task

Now, you might be wondering how to choose the right model tasks. Ask the following guiding questions to select the most appropriate LLM task for your specific business scenario. This structured approach helps reduce

ambiguity and ensures that your prompts trigger the right type of model behavior, increasing accuracy and integration readiness:

1. Do you need the original text that wasn't in the input?
2. Use text generation. If your goal is to create content that expands on or introduces new ideas not found in the input, such as writing a product description or composing an email response, then text generation is the right choice. It enables the model to synthesize information, emulate tone, or personalize messages.
3. Do you need to convert input from one form to another?
4. Use transformation. If you're translating between formats (e.g., plain English to SQL, bullet points to email), the transformation preserves the core content while adapting the format, tone, or structure. This approach is ideal for workflows that require standardization, localization, or format-shifting.
5. Are you pulling specific fields from unstructured text?
6. Use extraction. When working with dense, unstructured data (e.g., clinical notes, invoices, contracts) and you need to extract specific attributes into a consistent structure, such as JSON or CSV, extraction is the ideal approach. It's particularly important for structured analytics, automation, or integration with databases.
7. Do you want to assign a category or label to the input?
8. Use classification. Use this when your goal is to route or filter input into one of several predefined options, such as tagging sentiment, categorizing an email, or determining the topic of a document. This task is well-suited to building logic flows, triggering business rules, or summarizing input types.

Table 2-2 shows some common use cases. Each prompt should align with both the technical capability of the model and the practical needs of the system consuming the output.

Table 2-2. Common use cases and recommended LLM tasks

Use Case	Desired Output	Recommended LLM Task	Prompt Format Suggestion
Customer service routing	Category label	Classification	`Categorize this message as: billing, technical, or general.`
Financial KPI extraction	Structured JSON	Extraction	`Extract revenue, profit, and margin from this report in JSON.`
Legal clause transformation	Rewritten clause	Transformation	`Rewrite this clause in plain English for a layperson.`
Product description generation	Natural language text	Text generation	`Write a brief product description for this camera.`
SQL query from natural language	SQL statement	Transformation	`Convert the following question into a SQL WHERE clause.`
Email intent detection	One-word intent (e.g., inquire)	Classification	`Identify the intent of this email: {email_text}`

Best practices for prompt-task matching

Designing an effective generative AI system involves more than just selecting a task type; it also requires considering the specific characteristics of the task. Even with the correct classification or extraction strategy, poor prompt-task alignment can lead to unreliable outputs. Certain best practices help ensure that model behavior remains predictable, reproducible, and

well-integrated into your business pipeline. Consider the following guidelines to make your prompt-task pairings robust and production-ready:

Start with the desired output format

Before writing your prompt, consider what the receiving system expects. Will the output feed into a user interface, an analytics dashboard, a reporting engine, or a downstream API? For instance, a chatbot may need human-readable sentences, whereas a billing system may require JSON objects. Working backward from the expected output allows you to constrain your prompt more effectively and ensures that the format is immediately usable. Will the output feed into a UI, a SQL engine, or an analytics dashboard? Align the prompt design to produce that output type.

Clarify downstream consumers

You should always know who or what will consume the model's output. Is it a human support agent, a financial analyst, or a backend service? A report intended for stakeholders might benefit from using structured bullet points and domain-specific terminology. On the other hand, automated systems might require machine-readable formats with exact key-value pairs. This knowledge will help shape not only the format but also the level of detail, tone, and even vocabulary used in your prompts. Understand who or what uses the model's output. If it's an API integration, structured formats like JSON are often required. If it's a human reviewer, well-formatted text or bullet points may suffice.

Differentiate between open-ended and closed-ended tasks

An open-ended task, such as generating a creative ad slogan, demands a different strategy than a closed-ended task, like labeling sentiment. Open-ended prompts benefit from rich

context, tone cues, and few-shot examples to help steer the model toward stylistic or domain alignment. Closed-ended prompts, on the other hand, require strict constraints, clear format expectations, and sometimes even schema validation rules to avoid hallucinated or malformed output. Open-ended tasks (like text generation) thrive on few-shot examples and tone-setting context. Closed-ended tasks (like classification or extraction) require a tighter structure and often benefit from explicit formatting instructions.

Make your intent explicit

Models interpret language probabilistically. If you are vague in your instructions, the model may interpret it in ways you didn't intend. For example, saying *Summarize the ticket* is open to wide interpretation—does the user want a headline, a paragraph, a bullet list, or key fields? Instead, specify: *Return a JSON object with fields: issue_type, urgency, and resolution_steps*. The clearer your intent and constraints, the more predictable and reproducible the model's behavior will be. The model should never have to guess your intent. Specify what to extract, how to format it, and what the output should contain. This reduces ambiguity and improves reproducibility.

Iterate and test with edge cases

A prompt that works in ideal conditions may break when confronted with real-world variability. Always test across a wide range of inputs, including missing values, poorly formatted data, contradictory information, and edge-case phrasing. This testing helps uncover failure modes early, ensures the model is robust across various conditions, and enables you to optimize prompt design and fallback strategies. Use prompt evaluation frameworks or Databricks notebook logging to benchmark consistency across test

> cases. Always run your prompts against varied inputs, including incomplete, ambiguous, or borderline examples. This strategy helps validate whether your prompt and task match will hold under real-world conditions.

Tools like LangChain and Databricks Prompt Engineering UI allow you to build reusable, task-specific prompt modules and chain them together. For example, a pipeline might begin with a classifier to identify the request type, send specific inputs to an extractor for field parsing, and then transform the output into a readable summary. Matching the task to the prompt ensures reliability, modularity, and clarity at each step of the pipeline.

Aligning prompts to use cases

Prompt-task alignment refers to the practice of matching the model's behavior (task type) to the specific goal of a business use case. For example, if your objective is to extract structured data from unstructured text, you must use an extraction task with a prompt tailored to that need. Misaligned prompts—such as requesting a summary when a structured output is needed—can yield irrelevant or unusable results. Aligning the prompt to the task ensures the model delivers consistent, actionable, and format-compliant outputs that meet the expectations of downstream systems or users.

Let's consider the importance of prompt-task alignment with a healthcare example. Let's say you work with a healthcare team analyzing clinical notes. A prompt reads as follows:

```
Summarize this clinical note.
```

The model may generate a natural language paragraph that includes observations and assessments. However, if your goal is to extract specific ICD-10 diagnosis codes, this is not helpful. Instead, the correct prompt and task would be more like this:

```
Extract all ICD-10 diagnosis codes from the following note. Return
them in a JSON list under the key diagnosis_codes.
```

By aligning the model task (extraction) with the use case (code retrieval), you get consistent outputs that are easier to validate and use downstream.

Task type decision matrix

Each business use case typically maps to a specific model task type—such as classification, extraction, or transformation—and demands a carefully tailored prompt format. Choosing the right task and prompt structure is critical to producing outputs that are consistent, actionable, and aligned with downstream requirements. Table 2-3 helps to decide common enterprise use cases with the correct LLM task type as a quick reference to help determine which task type and prompt style best fit your use case.

Table 2-3. Task type decision matrix

Use Case	Desired Output	Task Type	Prompt Format Example
Support ticket routing	Category label	Classification	`Classify this ticket as: billing, technical, or general.`
Financial KPI extraction	JSON object	Extraction	`Extract revenue, expenses, and margin from this report in JSON.`
Clause simplification (legal)	Rewritten text	Transformation	`Rewrite this clause in plain English.`
Product description generation	Natural language	Text generation	`Write a brief product description for this item.`
Convert query to SQL	SQL statement	Transformation	`Convert this natural language query into a SQL WHERE clause.`
Email sentiment detection	One-word sentiment	Classification	`Identify the sentiment of this message.`

AI Pipeline Composition

Composing an effective AI pipeline means assembling modular, interoperable components that collaborate to transform user inputs into useful, structured outputs. These pipelines are foundational in enterprise-grade AI systems because they enforce repeatability, enable debugging, and

support governance. By orchestrating tasks like prompt formatting, model invocation, and memory management into discrete steps, you gain more control over the AI's behavior and performance. This approach is especially critical in regulated industries, high-volume use cases, and applications where auditability matters.

Designing effective generative AI workflows involves more than just crafting a clever prompt. In real-world production systems, you must build an orchestrated sequence of functional steps—known collectively as an AI pipeline. An *AI pipeline* is a structured flow of tasks where each component is responsible for a specific function, such as preparing input, formatting prompts, calling the LLM, parsing output, and preserving conversational memory.

To illustrate, consider a typical AI pipeline for an insurance claim assistant:

- The pipeline begins by accepting user input, such as a written description of a claim.
- A prompt template standardizes this input into a well-formed instruction.
- The LLM wrapper receives the instruction and forwards it to the large language model for processing.
- An output parser captures the model's response to ensure it is machine-readable (e.g., in JSON format).
- A context manager tracks the conversation state, allowing follow-ups to reference previous messages.

Think of this pipeline as a team of workers on an assembly line: one prepares the raw materials, another shapes them, another inspects the result, and yet another tracks the progress. Each step enhances the reliability, traceability, and scalability of your AI workflow.

This section breaks down these components in detail, demonstrates how they operate using LangChain, and explores best practices for assembling pipelines using Databricks tools.

Selecting Chain Components

A generative AI pipeline is composed of modular building blocks, where each component handles a distinct function in processing data and interacting with a language model. These responsibilities may include formatting prompts, executing model queries, parsing outputs, and managing session context. Much like a traditional software pipeline that validates inputs, processes logic, and formats outputs, an AI pipeline ensures that every step in the interaction with the model is controlled and reproducible. Choosing the right components—and assembling them in the right order—is essential for creating scalable, maintainable, and production-grade AI applications. This modularity enables easier debugging, targeted improvements, and safer deployment in real-world environments where consistency and transparency are crucial.

Core pipeline components

Core pipeline components form the backbone of any AI application. These modules handle tasks such as formatting prompts, executing model queries, parsing responses, and maintaining context across interactions. When combined, they enable reliable, scalable, and modular AI systems that can be reused and maintained across use cases.

Prompt templates

> These serve as structured blueprints for your prompts, transforming ad hoc instructions into standardized forms. Each template includes dynamic placeholders (e.g., {{user_query}}, {{product_type}}) that can be filled in programmatically. Using templates with dynamic placeholders makes prompt creation reusable and testable. Templates reduce errors by enforcing a known schema, help scale prompt logic across teams, and are especially effective when building applications like chatbots, fraud detection tools, or summarizers.

LLM wrappers/model runners

These modules interact directly with the underlying language model (e.g., OpenAI, Mistral, Cohere). They manage configurations such as:

- *Temperature:* It helps control randomness, which balances creative and conservative responses.

- *max_tokens:* For limiting output length, ensuring that the model doesn't return excessive or irrelevant information.

- *model_name*: To specify which version of the LLM to use. Wrappers enable retry logic, timeouts, logging, and monitoring, all of which are critical for ensuring production reliability.

Output parsers

These tools take the model's raw text output and transform it into structured formats, such as JSON, lists, or tables. This step ensures that downstream applications, such as business dashboards, analytics engines, or user interfaces, easily consume the data.

Memory or context managers

These components preserve context across user interactions. Memory types include the following:

- *Buffer memory:* Buffer memory stores the last few interactions in their original form, preserving the exact wording of recent exchanges between the user and the AI. This type of memory is ideal for short conversations

or applications where precision in recalling the latest statements is critical, such as technical support bots or form-filling assistants. Since it does not summarize or alter the message, it ensures high fidelity in tracking immediate context, making it useful when follow-up queries rely heavily on the exact phrasing of previous inputs.

- *Conversation summary memory:* This memory summarizes previous messages to save space while preserving the original intent. This type of memory condenses previous dialogue into a concise summary that retains key points and decisions from earlier turns. It is especially useful in long conversations where tracking exact wording isn't necessary, but understanding the overall topic and intent is. For example, a healthcare triage chatbot might summarize a patient's ongoing symptoms and history in one or two lines, allowing the model to stay on topic without being overloaded by lengthy transcripts.

- *Entity memory:* This memory type tracks key named entities, such as people, dates, organizations, product names, or locations, mentioned throughout a conversation. It allows the model to retain specific facts and references across multiple turns. For example, in a customer support scenario, if a user mentions *John Smith* and *order #12345* early in the exchange, entity memory ensures those details are remembered and reused in follow-up responses or summaries. This capability enhances coherence, personalization, and task continuity

in applications like CRM chatbots, virtual assistants, and automated interviewers.

Another example: if the user first says, *Show me laptops under $1000* and then follows with, *How about with 16 GB RAM?* the memory system ensures the second request builds on the earlier one without losing context.

These pipeline components are assembled in series or conditionally branched depending on the application logic. Each serves to isolate functionality, improve maintainability, and enhance testability.

Example using pipeline components

Let's say a customer types: *Can you recommend a few wireless headphones under $100?*

Here's how each pipeline component might operate:

1. The prompt template fills in a blueprint:

```
                    Find products related to {{product_type}}
that cost under {{budget}} dollars.
```

   Here, *wireless headphones* and *100* populate the placeholders.

2. The LLM wrapper submits this structured prompt to the model and controls how creative or focused the response should be.
3. The output parser takes the list of recommended products and transforms it into a JSON array with fields such as *name*, *price*, and *rating*.
4. The memory manager keeps track of the last query. If the user later asks, *What about noise-cancelling ones?* the system remembers the original category and price context.

Summarizing Claims with LangChain

Let’s implement a simplified claims summarization workflow using LangChain to illustrate the concepts you learned in this section. You are working in an insurance organization and need to process incoming claims written in natural language. These could be user-submitted messages, such as *Slipped on a wet floor and reported back pain.*

Rather than passing this text directly to an LLM for general interpretation, you want a system that always returns a structured summary with keys like `claim_type`, `urgency`, and `issue`. This strategy ensures consistency across responses, simplifies validation, and enables seamless downstream integration (e.g., into a dashboard, claims processor, or audit tool).

To build this, you’ll need the following:

- A prompt template that outlines the desired response structure.
- An LLM wrapper that handles interaction with the model.
- A chain that connects them and feeds the input to the model.

Here’s how that looks in the code:

```
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI
from langchain.chains import LLMChain

# Create a prompt template for summarizing insurance claims
summary_prompt = PromptTemplate.from_template(
    "Summarize this insurance claim and return a JSON with keys:
claim_type, urgency, and issue: {claim}"
)

llm = OpenAI(temperature=0)
summary_chain = LLMChain(llm=llm, prompt=summary_prompt)

# Run the chain with a sample input
result = summary_chain.run("Patient slipped on wet floor and reported
back pain.")
print(result)
```

The pipeline takes a free-text claim, converts it into a structured summary, and enables logging, auditing, or routing to human adjusters for further review. Figure 2-2 illustrates this process.

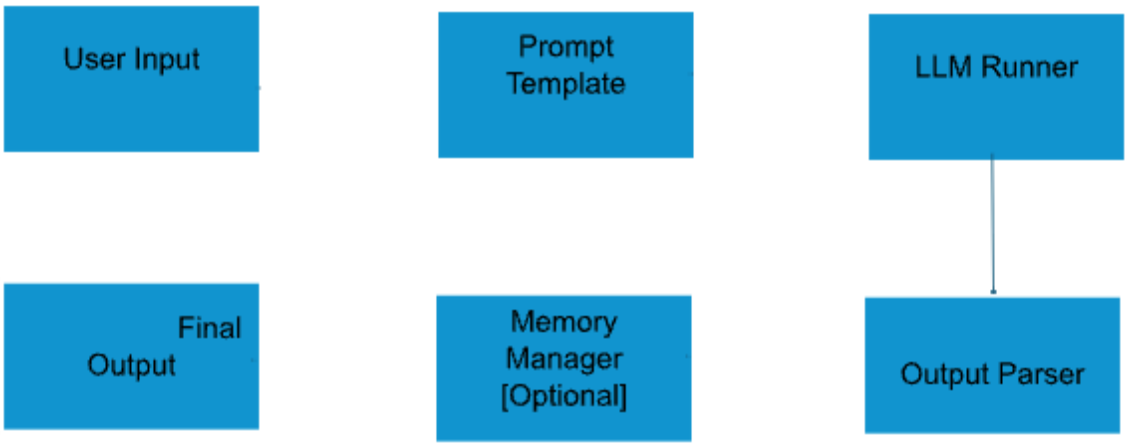


Figure 2-2. AI pipeline workflow

> **TIP**
>
> Avoid writing a single, comprehensive prompt that attempts to cover everything at once. Break tasks into smaller steps, each with its own component. This improves accuracy, reusability, and traceability.

Template Registries and Reuse

Managing pipeline elements at scale becomes increasingly important as organizations deploy more GenAI apps across departments. Without reusable assets and shared governance, prompt logic and workflow definitions become fragmented and difficult to maintain. That's where prompt registries come in—designed to support discovery, consistency, and versioning of reusable pipeline assets.

In enterprise teams, managing prompt templates and component logic at scale requires governance to ensure consistency, reusability, and auditability. As organizations build more LLM-powered applications across various functions, such as customer service, compliance, and analytics, the number of prompts and chains can grow rapidly. Without a system for

organizing and maintaining these templates, teams risk duplication, inconsistencies, and difficulty tracking updates.

One effective solution is to centralize prompt templates in a shared *registry*—a version-controlled repository that stores standardized templates, tracks changes, and enforces access permissions. This registry functions like a library, allowing teams to search for, reuse, and test approved templates without reinventing the wheel each time. By tagging templates with metadata such as `domain`, `task type`, or `version`, large teams can scale their prompt engineering practices while maintaining high quality and traceability.

Pseudo-UI for Prompt Registry

These registries promote reuse by enabling teams to share proven templates across projects and departments, thereby facilitating efficient collaboration. Version control allows developers to track changes, roll back to earlier states, and understand how prompts evolve.Registries also streamline testing by integrating with automated pipelines to validate prompt behavior across diverse inputs. Altogether, this infrastructure ensures that pipelines remain reliable, easier to debug, and aligned with compliance or quality standards, especially as the scale and complexity of AI systems grow. Table 2-4 shows a sample prompt design registry.

Table 2-4. Sample prompt design registry

Template Name	Description
summarize_claim_v2	Insurance claims
detect_fraud_scenarios	Fraud detection
support_ticket_classifier	Customer service

A well-designed AI pipeline breaks down complexity into manageable parts. By composing prompt templates, LLM wrappers, output parsers, and memory modules, you create systems that are easier to maintain, more interpretable, and better aligned with real-world workflows. LangChain and Databricks tools make these compositions easier to implement across diverse enterprise scenarios.

> **WARNING**
>
> Avoid monolithic prompt chains that try to do everything in one shot. Break down complex flows into modular parts to improve accuracy, reusability, and debuggability.

Translating Use Cases into Input/Output Design

Translating business use cases into a well-structured AI pipeline begins by identifying and formalizing the system's expected inputs and outputs. This step is foundational because the effectiveness of your AI solution hinges not only on what the model does but also on how it receives data and produces formatted outputs for consumption. You can think of this process as building a bridge between human understanding and machine processing: the model must understand the input in a way that it can comprehend and produce the output in a structure that downstream systems or users can act upon.

This input/output (I/O) formalization ensures that your AI workflow is logically coherent, consistently structured, and fully interoperable with any downstream systems, such as business intelligence dashboards, CRM software, rule engines, or even human-facing applications. Without this clarity, your pipeline might produce technically valid results that are unusable in context, leading to breakdowns in automation, compliance failures, or poor user experience.

The goal is to map each use case to a clear, testable *I/O contract*—a structured agreement that defines the shape, structure, and content expectations for both the inputs the model receives and the outputs it

produces. This contract enables modular pipeline components to operate predictably, reduces ambiguity in prompt design, and provides a clear basis for evaluating model performance and business impact.

Input/Output Design Use Case

Without clearly defined inputs and outputs, even well-engineered AI pipelines can break down, resulting in unusable data, inconsistent results, or failure to integrate with existing tools. Defining your I/O design upfront makes it easier to do the following:

- Align with the expectations of business stakeholders
- Reduce ambiguity in prompt formulation
- Create test cases that validate performance
- Ensure compatibility with APIs, dashboards, and databases

Let’s walk through a concrete use case of an Insurance Claim Validator app in more detail. Imagine you’re part of a team at an insurance company tasked with reducing the manual workload of reviewing incoming claims. Today, human agents must read through scanned documents or typed descriptions, classify the type of claim, verify whether all required information is present, and determine if the claim complies with coverage rules. This process is time-consuming and prone to inconsistency.

An AI-driven system can streamline this by automatically parsing input documents, detecting key information, validating claim completeness, and offering clear next-step recommendations. The system must handle multiple formats and extract structured information from unstructured inputs, all while preserving compliance and auditability. The business objective, therefore, is to ensure high-throughput, consistent claim validation, with flagged exceptions routed to human reviewers.

To design this workflow, we will perform the tasks in the following subsections.

Decomposing the task into multiple steps

Decompose the task as follows:

1. Utilize optical character recognition (OCR) or document AI to extract text from scanned documents. This process involves using pre-trained models or services to digitize and clean text from PDFs or image files. The extracted text forms the raw data that feeds into the next steps.
2. Classify the claim (e.g., auto, medical, property). This step uses a classification model or prompt-based logic to assign a category to the claim, which may affect how validation rules are applied later.
3. Validate the content against rules (e.g., coverage, missing fields). Based on the claim type, a set of business rules or regulatory criteria is applied. This validation process may involve checking for required fields, coverage eligibility, and data inconsistencies.
4. Recommend next steps (e.g., approve, escalate, request more info). The final stage involves applying logic—either LLM-based or rule-based—to suggest the next course of action. The output might include a recommendation flag, confidence score, or specific instructions for a human reviewer.

Define the inputs

Define the inputs as follows:

Raw text from scanned documents

> Typically pulled using OCR or document AI tools, this text includes unstructured content, such as medical notes, accident descriptions, or property damage details, that must be parsed and interpreted.

Metadata

Includes fields such as `claim ID`, `submission date`, `customer ID`, and `document timestamps`. These contextual signals help the AI system anchor its reasoning and make connections across different data fields.

Optional structured form entries

If available, this includes drop-down selections, checkboxes, or pre-filled values submitted through digital forms. These provide high-confidence data points that can complement or validate the information extracted from unstructured text.

Specify the outputs

Specify the outputs as follows:

A structured JSON object—such as the one shown next—helps standardize outputs for downstream processing, validation, and visualization. In enterprise environments, this structure is crucial because it enables various components—such as APIs, analytics dashboards, or workflow engines—to consistently and reliably consume model outputs.

Each designed JSON field represents an actionable insight or data point relevant to the insurance validation process. For instance, `claim_type` aids in routing logic, `is_valid` is crucial for compliance checks, `issues_found` helps flag anomalies for human review, and `recommendation` provides operational guidance. By clearly defining these fields in advance, you ensure that the AI system not only performs its task but also communicates results in a structured and actionable way.

Here is the JSON:

```
{
  "claim_type": "medical",
  "is_valid": true,
  "issues_found": ["missing policy number"],
  "recommendation": "Request policy number from customer"
}
```

Map to pipeline components

Map to pipeline components as follows**:**

Document parser for OCR

> This component is responsible for digitizing the contents of scanned claims and extracting unstructured textual data. It utilizes tools such as Azure Form Recognizer, Google Document AI, or the open-source Tesseract to convert images or PDFs into machine-readable text.

Prompt Template and LLMChain for claim classification

> After text extraction, this step involves crafting a prompt that guides a language model to identify the type of claim, such as *auto, medical, property*, etc., based on specific keywords, patterns, or contextual information. It feeds into a LangChain LLMChain or a custom classification model that outputs a structured label.

Rule-based filter for validation

> This module applies deterministic business rules or heuristics to the extracted and classified information. For example, if a medical claim is missing a required diagnosis code or if a property damage claim doesn’t specify the location, it flags the issue for review. This layer ensures consistency and enforces compliance logic.

Summary generation with prompt template

In the final phase, the system uses a tailored prompt to summarize its findings—such as validation status, detected anomalies, or rule violations—and generate actionable guidance. For example, it might confirm that a claim is valid but missing a policy number, then recommend asking the customer for that information. This step ensures the system communicates results to either a human reviewer or another automated process for final

decision-making or escalation, such as requesting additional details or recommending approval. The summary is typically presented to a human agent or passed to a downstream system for further processing. You can prototype and validate this flow using Databricks notebooks. Prompt chaining tools, such as LangChain, or orchestration platforms like MLflow Pipelines, allow you to modularize each step and fine-tune prompts, models, and validation logic incrementally.

Tips for Robust I/O Design

Translating a business use case into an AI pipeline isn't just about choosing a model—it's about engineering reliability in every step. The following best practices help ensure your input and output handling is consistent, transparent, and production-ready:

Design for failure modes

> Always anticipate edge cases—such as missing fields, invalid formats, inconsistent date values, or overlapping categories. Build your pipeline to detect and gracefully handle these scenarios by including fallback mechanisms or default behaviors. For example, if a required field is missing, flag it for review instead of letting the system fail silently. This process increases fault tolerance and enhances the system's trustworthiness.

Log intermediate steps

> Capturing intermediate input and output at each step of the pipeline, such as post-OCR extraction, post-classification, and post-validation, enables you to audit decisions, trace back the root cause of issues, and continuously improve your AI system. It also supports better debugging and compliance, especially in regulated industries where audit trails are essential.

Standardize output schema

Define and adhere to a fixed output schema with clearly named keys, using conventions such as camelCase or snake_case. Standardization ensures seamless compatibility with downstream systems, including APIs, business dashboards, and monitoring tools, thereby facilitating efficient integration. It also enables schema validation, which helps prevent runtime errors caused by unexpected field formats or names.

Visualize the flow

Create a visual representation of your I/O logic to enhance team communication and foster stakeholder alignment. Use tools like Mermaid.js for Markdown-based diagrams, Lucidchart for detailed flowcharts, or built-in Databricks visual components. Visualizations clarify dependencies and help identify opportunities for optimization within the pipeline.

> **TIP**
>
> Work backward from your desired output. Knowing exactly what form your output must take—such as a specific JSON schema or SQL row—makes it easier to design prompts, select tools, and structure inputs to align with those goals. By formalizing input/output design for each use case, you not only make your pipeline more testable and maintainable, but you also align it more closely with real-world business requirements. Always test your pipeline with edge cases, such as missing data or ambiguous inputs. This helps you refine prompt clarity and improve robustness before deployment.

Designing Reusable Prompt Templates

To get consistent results from a language model, you must use prompts that follow a well-defined structure. Ad hoc instructions often yield unpredictable or subpar outputs. In contrast, reusable prompt templates enable you to standardize your communication with models across various

tasks, teams, and products. This consistency not only improves performance but also streamlines collaboration.

A reusable prompt template serves as a blueprint that guides the model to respond predictably and coherently. It enables teams to reuse and modify a common format rather than writing prompts from scratch each time. These prompt templates are particularly useful in enterprise environments where compliance, brand voice, or response structure matters. On the Databricks Certified Generative AI Associate exam, you'll need to identify the components of a prompt template and evaluate their effectiveness in real-world deployments.

These templates enhance output quality by reducing variance, ensuring consistency in tone and formatting, and making model behavior easier to interpret, troubleshoot, and refine. They support a wide range of tasks, including customer query resolution, document summarization, data transformation, and code generation. In complex workflows, reusable prompts also make it easier to integrate models into automated systems or chains of operations without the need for constant rewriting.

Effective prompt templates typically include the following elements:

Instruction

> This is a clear and concise directive that tells the model exactly what action to perform. It sets the purpose and tone for the response. For instance, prompts like *Summarize the following review* or *Classify the sentiment of the text as Positive, Neutral, or Negative* provide the model with a specific task.

Context or example(s)

> These are additional cues or demonstrations that help the model understand the expected output. In few-shot prompting, you might show two or three example inputs and outputs to guide the model's behavior. This process boosts performance for specialized or nuanced tasks.

Input

> This refers to the unique content or query that the user provides at runtime. It could be a customer complaint, a product description, a data snippet, or any other dynamic content. A reusable template will swap this content into a designated placeholder.

Format or output constraint

> This specifies how the model should structure its response. It may include style instructions (*Write in a professional tone*), formatting rules (*Use bullet points*), or data types (*Respond in JSON with keys for title and summary*). Output constraints make integration with downstream systems more predictable.

For example, in a retail chatbot scenario, you might create a reusable template like this:

```
Instruction: Provide a helpful and friendly answer to a customer's
question.
Example:
Customer: "Do you offer free shipping?"
Answer: "Yes! We offer free shipping on orders over $50."
Customer: {{question}}
Answer:
```

You can fill in the variable *{{question}}* with dynamic input from a live system. Because the structure remains consistent, you can reuse this template to power many different queries while maintaining a uniform tone and format.

> **TIP**
>
> Keep prompts modular. You can combine building blocks (like examples, instructions, and formats) to fit specific tasks. This combination makes your system more maintainable and flexible as use cases evolve.

Reusable prompt templates act like the API contracts of LLMs. They help you frame the model's "interface" in a way that stays consistent, understandable, and easy to update. As you build more complex applications in Databricks, prompt templates will become essential to quality control and scalability.

Multi-Stage Reasoning Tools

When solving complex tasks, a single prompt often overwhelms the model by asking it to perform too many operations simultaneously, such as extracting facts, interpreting them, and presenting conclusions. This results in vague, shallow, or error-prone responses. For instance, if you prompt an AI to *Read this quarterly report and write a detailed summary of its financial implications*, you're effectively bundling four or five tasks into one: reading, understanding, extracting key data, analyzing trends, and summarizing. The model may respond inconsistently or miss critical insights because it generalizes across multiple mental steps.

Now consider a structured, multi-stage approach with three prompts:

- Extract financial metrics using a prompt tailored for extraction.
- Analyze the extracted metrics to identify patterns or trends.
- Summarize the insights using a prompt designed for clear, investor-friendly language.

Each prompt does one thing well. By narrowing the model's scope at each stage, you increase reliability, transparency, and reusability. You can inspect and validate outputs at every step, quickly identify errors, and reuse

components across different workflows. This technique, known as *multi-stage reasoning*, mirrors how people solve problems—by progressing step-by-step with a clear focus.

This section introduces two core strategies for implementing multi-stage reasoning:

- Tool ordering**,** where you explicitly define each stage in a fixed sequence.
- *Agent planning***,** where dynamic decisions determine the flow based on context

Both strategies appear in the Databricks Certified Generative AI Associate exam and are foundational to building scalable, modular LLM systems.

Tool Ordering for Sequential Logic

Tool ordering is the process of arranging tools or prompts in a logical sequence. When you complete tasks in the right order, your workflow produces clear and dependable results. Think of it like a recipe: if you try to frost a cake before baking it, the outcome will fail.

In AI, tool ordering is critical. For example, consider building a financial assistant that answers questions about a company's performance:

1. Search recent earnings PDFs using a document retriever. This stage ensures that you gather the most relevant and current documents based on user input or keywords. The retriever acts as a search engine, pulling in documents from a local repository or cloud source to feed into the next stage.
2. Extract financial data using a document parser or an LLM. This step isolates structured data such as revenue, expenses, net income, and growth metrics from unstructured text. You can use pattern-based parsing or prompt an LLM to extract only the specific values you care about, turning messy reports into clean, usable data.

3. Summarize the extracted data using a language model. With the structured information in hand, the final stage generates a clear, human-readable summary that interprets the results. This summarization makes it easier for end-users, such as analysts or investors, to understand the key takeaways without needing to review the full reports.

Each stage depends on the previous one. If you reverse the order, the model won't have the necessary context. Using tools like LangChain or Databricks notebooks, you can automate this sequencing to enforce the correct logic.

> **TIP**
>
> Use SequentialChain or RouterChain in LangChain to control flow based on task type or input characteristics.

Here's how you might implement the preceding in Python using LangChain:

```
from langchain.prompts import PromptTemplate
from langchain.llms import OpenAI
from langchain.chains import LLMChain, SimpleSequentialChain

# Step 1: Extract key financial terms
extract_prompt = PromptTemplate.from_template(
    "Extract the key financial terms from this report: {input}"
)
extract_chain = LLMChain(llm=OpenAI(), prompt=extract_prompt)

# Step 2: Summarize findings
summary_prompt = PromptTemplate.from_template(
    "Summarize the implications of the following financial terms:
{input}"
)
summary_chain = LLMChain(llm=OpenAI(), prompt=summary_prompt)

# Chain the tools in order
chain = SimpleSequentialChain(chains=[extract_chain, summary_chain],
verbose=True)
```

```
# Run the full process
result = chain.run("The Q4 report showed increased revenue but lower
net margins.")
print(result)
```

This script demonstrates how `SimpleSequentialChain` executes one step after another, ensuring that the output from the first prompt is passed directly into the second. Each chain utilizes a shared input format, providing seamless integration.

> **TIP**
>
> Use `SimpleSequentialChain` for straightforward, linear flows. For more control over variable names, branching, or error handling, consider `RunnableSequence` from `langchain_core`.

In this example:

- The first prompt extracts financial terms from a document.
- The second uses those terms to create an executive summary.
- LangChain ensures that these steps are executed correctly and in sequence.

This process reflects real-world use cases in AI pipelines: *extract → reason → respond.* Whether you're working in finance, healthcare, or customer service, multi-stage reasoning enables you to combine accuracy, traceability, and flexibility in your solutions.

Planning Agent Chains

Some problems don't follow a fixed, linear path. In real-world situations, such as troubleshooting or research, you often can't predict how many steps are needed or which tools the system should use. That's where *planning agents* come in. Planning agents combine reasoning with tool selection to make dynamic decisions about what to do next, step by step.

A planning agent works like a smart assistant. It begins with a goal, considers the available tools as needed, and stops until you get a satisfactory answer. This process makes it ideal for tasks that involve branching logic, information lookups, or dynamic decision-making.

Take a telecom scenario, for instance. Imagine a network-monitoring AI assistant responsible for diagnosing service outages across regions. When triggered by a user prompt, such as *Find the cause of the outage in New York*, the agent doesn't follow a fixed set of rules. Instead, it evaluates the situation step by step based on available tools and evidence it uncovers.

It starts by examining recent outage logs. If it finds log entries that match the specified timeframe or region, it proceeds to the next step. The agent then queries a user complaint database to identify any corroborating reports from affected customers. If the cause remains unclear, such as if the logs are inconclusive, it checks external weather data for that location and time to determine if adverse conditions (like a storm or flooding) may have played a role. Finally, it assembles a root cause analysis in natural language and presents it back to the user after gathering enough supporting evidence.

This process is not hardcoded. The agent uses its reasoning capabilities to decide whether to proceed, skip, or combine steps depending on what information it retrieves. This flexibility is especially valuable in dynamic, real-time environments, such as telecom, where causes vary, and data availability can differ from case to case.

Rather than hardcoding this sequence, a planning agent decides which steps to take based on what it finds. This process allows it to adapt the flow dynamically, just like a human expert would. Figure 2-3 shows the difference between sequential tool ordering and planning agent reasoning.

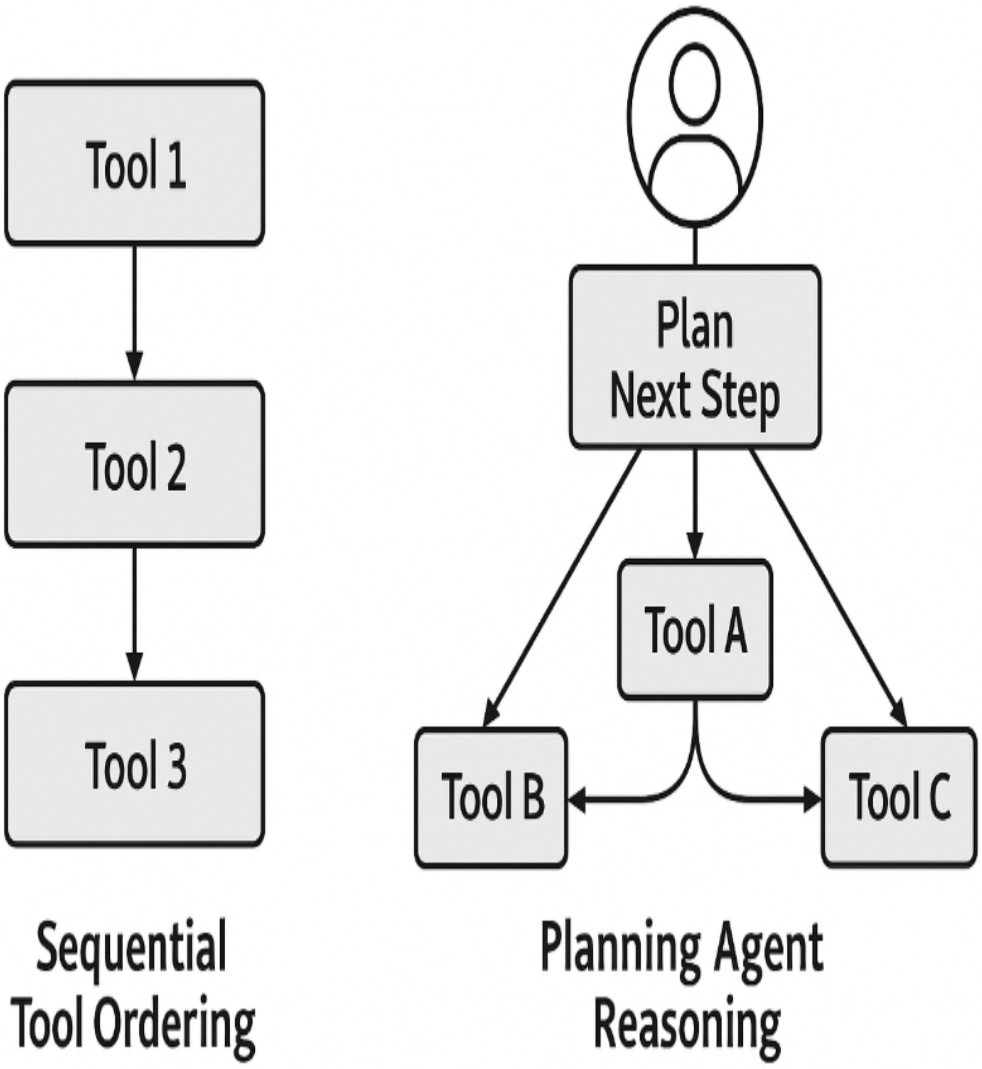


Figure 2-3. Sequential tool ordering vs. planning agent reasoning

Agent Planning with LangChain

LangChain supports agent-style workflows using patterns like *ReAct* (Reason + Act) and *Plan-and-Execute*. These patterns let the agent do the following:

1. *Receive a natural language task:* The agent starts by interpreting the user's request, such as *Find the cause of the outage in New*

York. It transforms this into an internal goal that will guide its reasoning process.

2. *Plan which tools to use and in what order*: The agent reviews the available tools and considers which are most appropriate to solve the problem. It evaluates what kind of information it needs first (e.g., logs) and what it might need next (e.g., user reports or weather), and organizes a possible sequence of tool invocations.

Execute those tools as needed: Guided by its plan, the agent calls each tool step by step. After each execution, it reassesses the situation to determine whether it requires additional information or can generate a final response. This loop continues until the agent completes the task or determines no further action is required. The loop is as follows: Reason, act, plan, execute. You can run these agents in Databricks notebooks by integrating LangChain tools and logic. Let's walk through a simple example using LangChain's ReAct framework to build a planning agent:

```
from langchain.agents import initialize_agent, Tool
from langchain.llms import OpenAI
from langchain.agents.agent_types import AgentType

# Define tools the agent can use
def search_logs(query):
    return f"Found matching log entry for: {query}"

def check_weather(location):
    return f"Current weather in {location} is clear."

tools = [
    Tool(
        name="SearchLogs",
        func=search_logs,
        description="Use this to search system logs for outages."
    ),
    Tool(
        name="WeatherChecker",
        func=check_weather,
        description="Use this to check the current weather
conditions."
    )
]
```

```
# Initialize the agent
llm = OpenAI(temperature=0)
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

# Ask the agent to investigate an outage
result = agent.run("Find the cause of the outage in New York region")
print(result)
```

Let’s look at how this code works step by step:

1. The agent receives the goal: investigate a New York outage.
2. It analyzes the user’s prompt to understand the objective and identifies *New York* as the target location and *outage* as the core problem.
3. It then evaluates which tools are most relevant. If system logs are available, the agent may begin by searching them for matching entries in the specified region.
4. If it finds useful logs, it determines whether further investigation is needed by checking if user complaints corroborate the system data.
5. When the evidence remains inconclusive, it proactively selects additional tools, such as checking real-time weather conditions, to identify environmental causes.
6. After synthesizing all retrieved data, the agent forms a comprehensive understanding and generates a clear, context-aware explanation of the likely cause of the outage.
7. It stops once the final output satisfies the original query or when further steps no longer add value.

This decision-making loop—observe, choose, execute, reassess—makes planning agents effective in adaptive, data-driven workflows. This mirrors how real-world agents operate in fields such as customer support, IT operations, or security triage.

> **TIP**
>
> Planning agents provide flexibility and adaptability. Use them for problems where you can't define the path upfront. Agents add power but also introduce complexity. Monitor for edge cases, infinite loops, or misuse of the tool. Use logging and timeouts to maintain reliability.

Planning agents enable your LLM applications to transition from scripted flows to intelligent behavior. As you build advanced pipelines in Databricks, mastering this pattern will help you design scalable, robust, and context-aware solutions. Multi-stage reasoning enables better control, interpretability, and task decomposition in large language models (LLMs) workflows. You've learned how to implement both *sequential chains* and *dynamic planning agents* using LangChain in Databricks.

In the next section, you'll go one step further and design *multi-agent systems*, where multiple agents collaborate, specialize, and hand off tasks to achieve complex goals efficiently.

Designing Multi-Agent Systems with Tool Use

As tasks grow more complex—such as end-to-end business processes, cross-domain decisions, or multi-step analytics—it becomes impractical to expect a single agent to manage all decisions, tools, and reasoning paths. That's where *multi-agent systems* become essential. In this architecture, multiple agents are assigned specialized roles, each equipped with specific tools and logic to handle a part of the overall task.

Designing multi-agent systems involves creating a collaborative ecosystem where agents can work independently yet interoperate seamlessly. For instance, in an AI-powered business workflow, you might assign one agent to gather financial reports, another to interpret them for risk exposure, and a third to generate recommendations. Each agent contributes a specific function, but collectively, they create a more intelligent and responsive pipeline. These agents pass context-rich data and decisions from one stage to the next, improving overall system performance, transparency, and scalability.

This process mirrors how humans work in teams—breaking large goals into smaller, role-specific tasks. In AI systems, multi-agent design facilitates the creation of reusable, modular, and adaptable workflows that can evolve without requiring a complete rebuild from scratch. Let’s understand this with an example:

Scenario: Insurance Claim Processing

Consider an AI system designed to automate complex insurance claim workflows. Rather than relying on a single, all-purpose agent to handle every aspect of the claim lifecycle, it’s far more efficient to distribute responsibilities across multiple specialized agents, each with domain-specific tools and logic:

Document agent

> This agent processes incoming claim documents, such as scanned PDFs or handwritten forms, extracting structured fields like claimant information, accident dates, and policy numbers. It ensures that noisy, unstructured text is converted into clean data.

Policy agent

> After the document agent completes its task, the structured data is passed to the policy agent, which verifies the extracted details against the customer’s policy. It validates

coverage, flags any policy violations or limitations, and confirms eligibility for further assessment.

Assessment agent

If coverage is valid, the data is passed to an assessment agent, which estimates the cost of repair or replacement. It may use internal rules, third-party APIs, or damage-assessment models trained on historical claim data.

Resolution agent

Once the assessment is done, the final resolution agent compiles all prior outputs and generates a natural language decision summary, such as an approval letter, payment estimate, or denial explanation.

These agents operate as a coordinated system. They pass data downstream but may also send it backward for revision if inconsistencies or missing information are detected, just like how a human team might ask someone to double-check the inputs before proceeding.

Here's a simplified illustration of how agents might interact in a coordinated system:

```
from langchain.agents import Tool, initialize_agent, AgentType
from langchain.llms import OpenAI

# Define basic tools each agent might use
def extract_claim_details(text):
    return "Claimant: John Doe, Damage: $1500, Policy#: 789XYZ"

def validate_policy(policy_id):
    return f"Policy {policy_id} is valid and active."

def assess_damage(details):
    return "Estimated repair cost: $1,500"

def finalize_resolution(assessment):
    return "Claim approved. Notification sent."
```

```
# Register as tools for agent use
tools = [
    Tool(name="ExtractClaim", func=extract_claim_details,
description="Extract data from claim documents."),
    Tool(name="CheckPolicy", func=validate_policy,
description="Validate policy coverage."),
    Tool(name="DamageAssessment", func=assess_damage,
description="Assess damage value."),
    Tool(name="ResolutionGenerator", func=finalize_resolution,
description="Create final claim decision message.")
]

# Set up controller agent
llm = OpenAI(temperature=0)
agent = initialize_agent(tools=tools, llm=llm,
agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, verbose=True)

# Run a coordinated workflow
result = agent.run("Process an insurance claim for policy ID 789XYZ
with reported vehicle damage.")
print(result)
```

This example simulates a multi-agent architecture where each step uses a specific tool, and a controller coordinates execution. This type of system reduces cognitive load per agent, increases transparency, and allows you to update one agent's logic (e.g., pricing models) without changing the rest of the pipeline. Figure 2-4 shows this process.

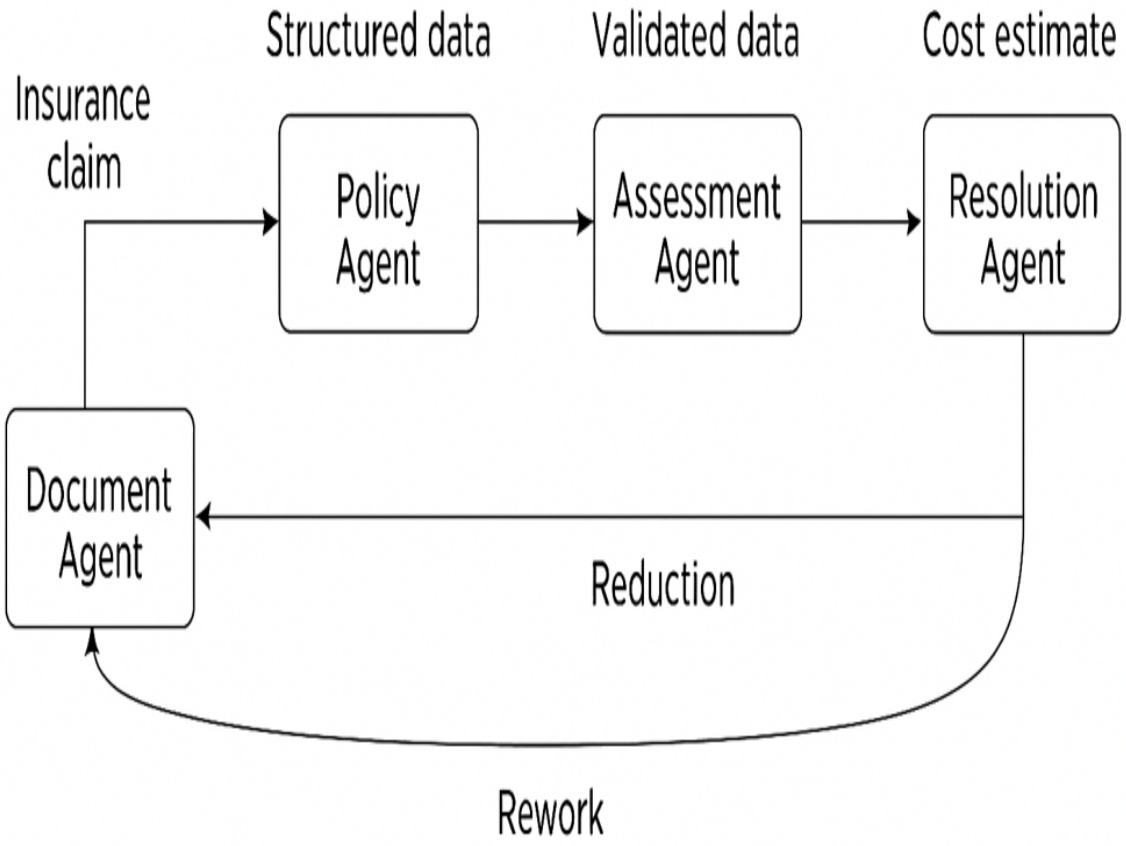


Figure 2-4. Insurance claim multi-agent architecture

[Note] This inter-agent communication requires shared memory, clear task boundaries, and a consistent data format. You can manage this using orchestrators like LangChain's AgentExecutor or shared context managers in Databricks notebooks.

Key Design Principles of Multi-Agent Systems

To build effective multi-agent systems, you need to apply principles that promote clarity in responsibilities, modularity in execution, and resilience in decision-making. These principles serve as a foundation to ensure each agent operates independently yet harmoniously within a larger system:

Define agent roles clearly

Every agent should have a well-defined and scoped responsibility. For example, one agent handles data extraction while another manages compliance checking. This modularity makes systems easier to maintain and allows for targeted improvements or debugging.

However, if you don't define agent roles clearly, overlapping responsibilities can lead to conflicts, redundant work, or data inconsistencies. For instance, two agents trying to process and validate the same document independently might return contradictory results or overwrite each other's outputs. It also becomes harder to identify where errors originate or how to isolate underperforming components. A lack of clear role boundaries renders the system a tangle of loosely coordinated behaviors rather than a streamlined, cooperative architecture.

Use consistent interfaces

All agents should communicate through predictable formats, such as structured JSON or validated schemas. This predictable format reduces integration errors, prevents misinterpretation between agents, and enables plug-and-play design when swapping or upgrading agents.

For example, imagine an assessment agent outputs damage estimates in a nested JSON structure, but the resolution agent expects a flat CSV-like dictionary. If these formats aren't aligned, the system could either crash or silently generate incorrect summaries. In contrast, using a shared schema, validated by both agents, ensures data integrity and consistent understanding.

Without consistent interfaces, agents may misread or fail to process each other's outputs correctly. These inconsistencies can result in dropped data, incorrect assumptions, or redundant error-handling logic. The lack of standardization breaks the chain of collaboration and makes debugging across agent boundaries extremely difficult.

Support asynchronous or iterative reasoning

The agent should be designed to operate independently, where possible. This process enables parallel processing—for instance, running a fraud

detection agent and a policy validation agent simultaneously. In some cases, agents may need to trigger one another recursively to refine results through iterative passes.

For example, in a financial workflow, a compliance agent and a risk evaluation agent can operate on the same transaction simultaneously. The compliance agent checks regulatory thresholds while the risk agent assesses exposure based on historical data. If either agent flags an issue, it could trigger a review agent that reruns earlier evaluations with stricter parameters. This not only speeds up the process but adds robustness through parallel verification.

Without support for asynchronous or iterative reasoning, agents are forced into a rigid sequence, even when steps could safely run in parallel. This leads to longer runtimes, underutilized resources, and difficulty scaling workflows for high-volume or real-time systems.

Use centralized coordination if needed

In more complex flows, use a controller agent or orchestrator to manage task routing, execution order, and dependency resolution. This coordination layer is essential when the workflow includes conditional logic, dependencies between agents, or real-time decisions that depend on multiple inputs.

For example, consider a multi-agent retail pricing system. One agent monitors competitor pricing, another checks inventory levels, and a third recommends discounts. Without a coordinator, these agents might operate out of sync, leading to a situation where the pricing agent drops a price below cost without knowing inventory is already running low. A centralized coordinator ensures that actions are sequenced logically, constraints are respected, and insights from one agent inform the others in real-time.

When coordination is missing, agents may duplicate work, contradict each other, or miss dependencies. This results in erratic outcomes, a poor user experience, and increased maintenance complexity. A well-structured

orchestrator prevents these issues by enforcing order and maintaining context across the full system.

By applying these principles, your multi-agent system becomes more scalable, interpretable, and resilient to faults. If one component underperforms or requires updates, you can address it independently without disrupting the entire system.

This structure enables your AI workflows to be more scalable, interpretable, and resilient to faults. If one agent fails or underperforms, you can isolate and refine it without having to rewrite the entire system.

Summary

In this chapter, you explored how to design effective Generative AI applications by aligning prompts, tasks, and workflows with real business needs. You learned how to craft structured prompts, match model tasks like classification or extraction to use cases, and build modular pipelines that separate responsibilities. Most importantly, you saw how multi-stage reasoning, through tool ordering and planning agents, helps break down complex workflows into manageable, repeatable steps.

Applying these techniques in tools like LangChain or Databricks notebooks allows you to build smarter, more reliable AI solutions that adapt to real-world data and decisions.

The next chapter dives deeper into how to prepare data for LLMs and RAG.

Practice Questions

Following are multiple-choice questions to reinforce your understanding of the concepts in this chapter. These are structured to reflect the style and expectations of the Databricks Generative AI Associate certification exam. The answer key appears at the end of the questions.

1. **1.** What is the main purpose of using a `PromptTemplate` in LangChain?

A. To speed up the LLM inference process

B. To control the token length in the LLM output

C. To define reusable, parameterized prompts with a consistent structure

D. To deploy models to a REST API

1. **2.** Which of the following use cases requires the *extraction* task type?

A. Generating a summary of a quarterly report

B. Classifying customer emails into support categories

C. Pulling fields like revenue and expenses from a financial document.

D. Rewriting a legal clause into plain English

1. **3.** What is the benefit of using a few-shot example in structured prompting?

A. It reduces latency in prompt execution

B. It teaches the model the desired output behavior using in-context examples

C. It eliminates the need for downstream output parsing

D. It allows the model to avoid schema definitions

1. **4.** In a multi-stage reasoning system, why is task decomposition critical?

A. It improves randomness in the generated output

B. It allows the system to skip all validation steps

C. It isolates each reasoning step for better accuracy and traceability

D. It reduces the number of tools needed in LangChain

1. **5.** In LangChain, what does `SimpleSequentialChain` enable you to do?

A. Execute prompts in parallel for performance

B. Run tools with dynamic ordering based on input

C. Enforce fixed step-by-step execution using prompt chains

D. Automatically validate schema compliance

1. **6.** What happens if you choose the wrong LLM task type for your use case?

A. The system defaults to classification

B. You may receive unstructured or irrelevant outputs

C. The prompt will be ignored by the model

D. LangChain will automatically correct the task type

1. **7.** Which principle is *not* part of effective multi-agent system design?

A. Shared memory between agents

B. Role clarity for each agent

C. Randomized tool execution

D. Standardized interfaces

1. **8.** What does the agent pattern ReAct enable within LangChain?

A. Real-time context memory storage

B. reasoning and tool calling in a step-by-step loop

C. Visualization of prompt chains

D. Summarization of multi-agent outputs

1. **9.** Why are prompt templates called the “API contract” for LLMs?

A. They enable testing through RESTful endpoints

B. They define static output schemas only

C. They allow consistent and expected interaction with the LLM

D. They store access credentials for model endpoints

1. **10.** You're asked to extract *claim_type* and *urgency* from unstructured insurance text. Which task and prompt style should you use?

A. Text generation with an open-ended prompt

B. Classification with a multiple-choice prompt

C. Extraction with a schema-defined prompt

D. Transformation using a plain-language template

Table 2-5. Answer Key:

Question	Answer	Explanation
1	**C**	`PromptTemplate` enables reusable, parameterized prompts with placeholders for consistent structure.
2	**C**	Extraction is used to extract structured fields, such as revenue and expenses, from unstructured text.
3	**B**	Few-shot examples guide the model to produce consistent outputs by showing desired behavior.
4	**C**	Task decomposition improves accuracy and traceability by isolating logical steps.
5	**C**	`SimpleSequentialChain` runs chains step-by-step in a fixed order, passing output to the next stage.
6	**B**	Misaligned task types result in outputs that are either irrelevant or unusable, such as prose instead of structured data.
7	**C**	Randomized tool execution would lead to inconsistent or chaotic agent behavior, ReActthereby violating system integrity.

Question	Answer	Explanation
8	**B**	ReAct agents “reason” and “act” in a loop, selecting and using tools step-by-step.
9	**C**	Prompt templates define the structure and expectations, much like an API contract defines Input/output (I/O).
10	**C**	Extraction with schema guidance (e.g., via JSON keys) is the correct pattern for structured data from free text.

Hands-On Lab: Multi-Agent Workflow with LangChain + OpenAI in Databricks

Scenario

You’re working with a company that handles auto insurance claims. Rather than using one large AI model to handle all tasks, you’ll create a modular pipeline composed of four specialized agents, each designed for a specific step in the claim process:

Document Agent

> Extracts structured data from a claim report stored in a Databricks table.

Policy Agent

> Validates whether the associated policy is active and eligible.

Assessment Agent

Uses an LLM to determine if the claim should be auto-approved or flagged for manual review.

Resolution Agent

Generates a final decision statement based on the previous step.

This agent-based pipeline approach reflects real-world enterprise deployments where interpretability, traceability, and modularity are essential.

Objective

By the end of this lab, you will:

- Understand how to configure LangChain tools in Databricks and chain multiple specialized tools using LangChain agents
- Create reusable functions and prompts for each step of the workflow
- Use LangChain's agent framework to coordinate tasks in a zero-shot fashion
- Observe how each agent contributes to the overall decision-making process and coordinate decisions like validation, risk assessment, and resolution
- Use OpenAI to reason through structured task steps

You will simulate the entire pipeline inside a Databricks notebook using PySpark for data setup and OpenAI for decision logic. This lab provides a practical foundation for building multi-step intelligent workflows using LLMs and agent orchestration tools.

NOTE

You can access this lab on Github at link.

Chapter 3. Preparing and Chunking Data for RAG Applications

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at *ccollins@oreilly.com*.

Imagine you're asking a large language model (LLM) to answer a question using a massive stack of documents. If those documents are disorganized, repetitive, or full of irrelevant details, the model may struggle to find the right answer, just as you would in a cluttered library. Retrieval-augmented generation (RAG) systems work similarly: the cleaner and more structured the data, the better the model performs. Rather than simply feed the model more content, the goal is to curate, filter, and format that content so it's useful at the moment of retrieval. In this chapter, you'll explore hands-on techniques for breaking up documents into manageable pieces (a process called *chunking*), cleaning noisy inputs, and structuring data for high-quality results. These are not just theoretical exercises—they're essential skills for passing the Databricks Certified Generative AI Associate exam and building effective AI systems in the real world.

Preparing data for retrieval in RAG involves more than basic preprocessing. In this context, *noisy inputs* refer to parts of a document that are irrelevant, redundant, or unstructured in a way that interferes with the model's ability to retrieve useful information. Examples include repeated paragraphs, formatting artifacts, headers, and footers, as well as unrelated content such as advertisements in web-scraped data. Removing or correcting these elements ensures that only high-quality, relevant data is used during retrieval. You'll need to strategically chunk documents, filter out redundant or noisy content, and convert the source material into structured formats suitable for efficient semantic search. These operations are foundational for building scalable and reliable AI systems, especially when working with enterprise data in Databricks environments.

Prerequisites

Before starting this chapter, make sure you're familiar with the following:

- What LLMs are and how they work at a high level
- The concept of model inference and common output types (e.g., text, structured data)
- Basic Python syntax and Databricks notebooks

Learning Objectives

After completing this chapter, you will be able to do all of the following:

- Implement document chunking strategies that optimize retrieval performance in RAG workflows.
- Apply filtering techniques to reduce noise and redundancy in source data.
- Convert and structure content into Delta format for efficient querying.

- Evaluate and tune retrieval quality using precision, recall, and reranking metrics.
- Troubleshoot gaps in retrieval by diagnosing data preparation issues.

Document Analysis and Chunking

To utilize RAG systems effectively, you must begin with properly structured input data. One of the most important early steps in this process is chunking, which involves dividing large documents into smaller, logically coherent sections. This division is significant because LLMs cannot process entire documents at once; they operate within a *context window* that limits the amount of text they can comprehend. If the text isn't chunked carefully, the model might miss key information or return irrelevant answers.

Chunking isn't just a mechanical or formatting task. It requires you to consider how a human reader would break down and comprehend the content. For example, a legal contract might be best chunked by clauses, while a user manual might work better when chunked by sections or step-by-step instructions. These decisions affect what parts of the document are retrieved in response to a query—and, therefore, whether the model generates a useful answer.

In this section, you'll learn how to analyze your source documents with an eye toward structure and purpose. You'll explore various chunking strategies—from simple fixed-length blocks to more advanced techniques like semantic chunking that rely on natural language patterns. You'll also see how tools like Python's spaCy library and Databricks notebooks can help automate chunking while keeping it aligned with your specific use case.

By the end of this section, you'll understand why chunking is not just a technical prerequisite but a strategic choice that can make or break the quality of a RAG-based application.

Chunking Strategies and When to Use Them

When working with documents in a RAG pipeline, it's essential to prepare them in a manner that allows a language model to comprehend them. One key step is breaking large documents into smaller, manageable sections—known as chunks. Language models have a limited capacity for processing text—the context window, as mentioned. Chunking ensures that each piece of the document is small enough to fit within that window while still carrying meaningful information.

Not all chunking is equal. There are various chunking strategies, each tailored to specific document types and use cases. Some prioritize simplicity, while others aim to preserve the natural flow or meaning of the content. Choosing the right approach requires understanding how your users will query the data, what kind of content you're working with, and what the language model needs to generate accurate responses. This subsection introduces these chunking strategies, compares their strengths and weaknesses, and explains how to decide which to use for your specific retrieval goals.

To help you evaluate each method clearly, the following sections outline each major chunking strategy, along with its use case, trade-offs, and a real-world example.

Fixed-Length Chunking

The simplest approach and one of the easiest to automate, fixed-length chunking divides documents into equal-sized blocks of uniform size based on a defined number of tokens, characters, or words. This method ignores sentence or paragraph boundaries, which means the chunks may start or end mid-sentence or mid-thought.

Let's examine where this strategy is best suited.

This method is particularly useful for documents that are consistently structured or where context is evenly distributed across the text, such as CSV files, log files, or records with repeating formats. For instance, if each

entry in a server log contains roughly the same kind of information, fixed-length chunks can facilitate fast and predictable processing.

Here's a real-world example: a cybersecurity team utilizes logs from firewall activity to train an RAG model, which helps analysts respond to potential threats. Since each log entry follows the same schema (for example, timestamp, source IP, destination IP, and action), breaking this data into fixed 50-line chunks enables the model to retrieve and examine relevant portions quickly. The uniform structure makes fixed-length chunking both reliable and scalable in this context.

But consider what you give up when choosing this strategy. While it's simple and fast to implement, fixed-length chunking carries a high risk of breaking up logical units of meaning. A sentence might be split between two chunks, making it harder for the language model to understand the complete context. This ambiguity can lead to poor retrieval relevance and weaker generative responses, particularly in content that relies on full sentences or coherent paragraphs.

Sentence-Level Chunking

Now let's look at a more linguistically aware method: sentence-level chunking. This approach breaks text into grammatically complete, sentence-sized pieces for clearer meaning and easier parsing. Here, each chunk contains a fixed number of complete sentences (for example, 3–5 sentences per chunk). This method ensures that each chunk forms a grammatically complete unit, preserving sentence structure and improving the readability of the retrieved content. It's especially helpful when working with text that already follows clear sentence formatting.

When is this approach a good fit? It works well for moderately structured content, such as news articles, FAQs, and internal knowledge base articles. These formats typically present information in short, standalone sections where individual sentences convey useful meaning without relying heavily on long-range context.

Imagine a company's customer support team maintaining a large internal FAQ document that covers common troubleshooting steps. When a support agent queries a RAG system for guidance on how to handle a login issue, sentence-level chunking enables the model to retrieve focused, standalone advice, such as *Reset your password using the account recovery link*, without mixing in unrelated steps. Since each chunk contains a few complete and self-contained answers, retrieval remains precise and on-topic.

There are limitations to this method. It does maintain grammatical integrity and avoids fragmenting sentences, which helps the model interpret each chunk more accurately. However, it may lose broader context if a concept spans multiple sentences across chunks. For example, multi-step instructions could be split in a way that leaves each chunk incomplete without the others.

Paragraph-Based Chunking

Next, we have paragraph-based chunking—a method that aligns naturally with the structure of many documents. In this approach, each *paragraph* becomes a chunk, assuming the document has clear paragraph separation. This strategy preserves the natural boundaries of thought that paragraphs represent, making it easier for the language model to interpret the information within context.

This strategy is ideal for well-formatted text such as academic papers, reports, and blogs. These document types often use paragraphs to separate ideas, methods, or arguments, which makes them excellent candidates for paragraph-based chunking.

A real-world example might involve a data science team using technical whitepapers to train a RAG-based assistant for internal use. Each whitepaper contains detailed sections on methodology, results, and analysis, organized by paragraph. By chunking the document at the paragraph level, the assistant can retrieve a specific explanation of a model's architecture or

a performance result without dragging in unrelated content. This keeps the response precise and tied to one conceptual unit.

Of course, there are downsides. Paragraph-based chunking is easy to implement and often produces human-readable results. But the sizes of paragraphs can vary greatly, leading to inconsistency in chunk length. Short paragraphs may lack sufficient context, while long ones could exceed the model's token limit, reducing the effectiveness of retrieval and generation.

Sliding Window Chunking

The sliding window chunking approach is a strategy that prioritizes continuity by intentionally overlapping content. Here, you create overlapping chunks to preserve context across boundaries. For example, a 500-token chunk with a 100-token overlap allows each new chunk to retain some of the context from the previous chunk. This method ensures that information split across chunk boundaries isn't lost during retrieval.

Sliding window is effective for question-answering systems or any scenario where ideas span across multiple sentences or paragraphs. It is particularly helpful in legal or technical documents where precision and continuity are critical.

A real-world example could involve a legal tech company building a RAG system to help users understand contract clauses. Legal language often spans multiple paragraphs and relies heavily on cross-references and precise phrasing. Using sliding window chunking, the system ensures that parts of a clause or definition that might be split at a boundary still appear in adjacent chunks. This preserves the meaning and increases the likelihood that the correct legal interpretation is retrieved.

Be aware of the trade-offs. Although sliding window chunking improves context retention and retrieval accuracy, it also increases the number of chunks stored and processed. This results in higher computational and storage costs, particularly for large-scale document repositories.

Semantic Chunking

Now we turn to the most advanced technique. Semantic chunking adapts to the content's meaning rather than relying on size or surface structure.

This advanced technique uses natural language processing (NLP) tools to detect natural boundaries in meaning—splitting at topic changes, heading levels, or discourse markers. Unlike simpler methods that rely on length or fixed structure, semantic chunking tries to understand what the text is about before deciding where to split it.

Semantic chunking is best for long-form unstructured content like transcripts, meeting notes, or books. These types of documents often contain shifts in topic or speaker that don't align with clean sentence or paragraph boundaries.

Consider a real-world scenario: a healthcare organization wants to use doctor-patient visit transcripts in a RAG application to help administrative staff extract key insights. These conversations often shift topics, starting with symptoms, then moving to history, and finally to treatment. Using semantic chunking, the system can segment the transcript into meaningful sections based on topic changes rather than sentence count. This allows the model to retrieve only the section related to medication recommendations, for example, without including unrelated small talk or examination details.

One tradeoff is that semantic chunking requires more computation and preprocessing, often involving embeddings and similarity scoring to detect semantic shifts. While this increases complexity and processing time, the resulting chunks tend to be the most contextually rich and useful for retrieval tasks involving complex, nuanced information.

> **NOTE**
>
> Semantic chunking often uses embeddings and similarity scores to detect topic shifts. Tools like spaCy or transformers from Hugging Face can help.

Table 3-1 compares the different shunking strategies with their pros and cons.

Table 3-1. Chunking strategies, pros and cons

Strategy	Best For	Pros	Cons
Fixed-Length	Logs, structured data	Simple, fast	May break context
Sentence-Level	Articles, FAQs	Preserves grammar	Limited broader context
Paragraph-Based	Reports, blogs	Human-readable, easy to chunk	Inconsistent sizes
Sliding Window	QA systems, legal documents	Maintains continuity	Resource-intensive
Semantic	Books, transcripts, complex docs	Meaningful, adaptive	Complex to implement

In practice, you'll often combine chunking techniques. For example, you might use paragraph-based chunking followed by a sliding window to handle variability in paragraph length. The key is to balance retrieval granularity with enough context for the LLM to generate a coherent response. For example, you could implement two of these strategies using open-source NLP libraries to create retrievable chunks from raw text.

Controlling Overlap and Granularity

When preparing documents for RAG, one of the most important design decisions involves how to divide your content into chunks. This involves two key parameters: *granularity*, which refers to the size of each chunk, and *overlap*, which describes whether adjacent chunks share some of the same content.

These decisions directly impact how well your system retrieves relevant information and how effectively an LLM can comprehend that information in context. For example, if your chunks are too large, they may contain extra, unrelated content that distracts the model. If too small, they may lack enough context for the model to generate a useful response.

Overlap becomes important when the meaning of the text spans across chunk boundaries. Without overlap, you might cut a sentence or paragraph in the middle, causing the model to lose track of the full idea. Adding some repeated content between chunks can help preserve continuity, though it also increases processing overhead and may introduce redundancy.

The trick is finding the right balance between chunk size and overlap based on the structure of your documents and the needs of your RAG system. These techniques are especially useful when working with documents such as technical manuals, research papers, or lengthy articles, where information is interconnected and cannot be easily separated.

Let’s begin by learning what granularity means in the context of chunking and how different levels affect retrieval. *Granularity* refers to the amount of content included in each chunk, typically measured in tokens, characters, or sentences. In the context of a RAG system, selecting the right chunk size is crucial because it determines the amount of context the model receives when answering a question. If the chunk is too large, it might include irrelevant or distracting content. If small, key pieces of information might be excluded, reducing the model’s ability to generate a useful response.

The ideal chunk size often depends on the structure, length, and density of the content:

- A product FAQ typically consists of brief, self-contained answers where each question and answer form a discrete, meaningful unit. This structure makes it a great candidate for fine-grained chunking, such as by individual sentences or question-answer pairs. By keeping each chunk small and focused, you ensure that the model receives highly relevant and unambiguous input during retrieval. For instance, if a user asks a question about refund policies,

retrieving a single FAQ chunk specifically answering that topic is more efficient and accurate than retrieving an entire document section. Smaller chunks, like these, also reduce noise and enhance the relevance of matches during similarity search, making them especially valuable in customer support and knowledge base applications.

- A legal contract, on the other hand, is composed of dense, interdependent clauses that are often legally binding and contextually linked. Each clause may refer to previous or subsequent sections, which means that splitting these arbitrarily could cause the language model to miss or misinterpret critical information. In such cases, using paragraph-based chunking helps preserve the integrity of the legal logic and terminology.

If you were to chunk a contract by individual sentences or tokens, a clause defining “termination conditions” might be separated from related clauses about “notice period” or “penalties.” This fragmentation would hurt both retrieval and comprehension. By chunking at the paragraph level, you capture complete legal ideas in a single unit, making it more likely that a retrieval system will return all necessary context for downstream analysis or generation.

Legal documents are a prime example of where *coarse* granularity—larger chunk sizes—is not only acceptable but necessary. The downside is that such chunks may approach or exceed model token limits, so careful evaluation and possible truncation strategies should be considered during implementation.

It’s also important to consider the model’s token limit when designing your chunks. Most LLMs, including those used in Databricks, have a maximum context window, typically ranging from 4,000 to 8,000 tokens, depending on the model version. If a chunk exceeds that limit, it may be truncated or rejected altogether, which can degrade the quality of the model’s responses.

At the same time, if your chunks are too small, they may not include enough information to answer the user’s query meaningfully. This results in

poor retrieval accuracy and less helpful outcomes.

> **TIP**
>
> The goal is to strike a balance: keep chunks concise enough to fit within the model's input limits, but rich enough in context to be useful.

When planning your chunking strategy, consider the total *context budget* the model has to work with and aim to maximize the relevance of each token included. Example 3-1 shows a basic example that splits a text into token-based chunks.

Example 3-1. Controlling chunk granularity using token-based splitting

```
 def tokenize_and_chunk(text, chunk_size=200):
    """
    Splits a given text into non-overlapping chunks based on token
count.
    This example uses a fixed granularity of 200 tokens per chunk.
    """
    import tiktoken  # Tokenizer compatible with OpenAI models

    enc = tiktoken.get_encoding("cl100k_base")
    tokens = enc.encode(text)

    chunks = []
    for i in range(0, len(tokens), chunk_size):
        chunk = tokens[i:i + chunk_size]
        if chunk:
            chunks.append(enc.decode(chunk))

    return chunks
```

> **TIP**
>
> Use a larger `chunk_size` (for example, 300–500 tokens) for documents that contain interrelated content like contracts or policies. Use smaller chunks (100–200 tokens) for short, distinct facts or frequently asked questions (FAQs). Start with chunks that fall in the 100–300 token range, then test and adjust based on retrieval performance and your application's needs.

Overlapping Chunks for Contextual Flow: Fixed and Dynamic

Overlap means that each chunk shares a portion of its content with the previous or next chunk. This technique is particularly useful in scenarios where information does not fall neatly into independent, isolated blocks. Many real-world texts—like academic papers, technical documentation, or product manuals—present ideas that unfold over multiple sentences or paragraphs. If you split this type of content into strictly non-overlapping chunks, the flow of information can get interrupted, leading to poorer comprehension by the model.

For example, imagine you are chunking a user manual for a complex software system. If one chunk ends with a partial description of a command and the next chunk starts with its usage example, the language model may miss the connection if the command itself wasn't repeated. By overlapping the final sentence or two of the first chunk with the start of the second, you preserve continuity and improve the model's ability to follow the logic.

Overlap helps ensure that each chunk is semantically complete—that is, that it contains enough self-contained meaning to make sense on its own, even if retrieved in isolation. This is especially important for retrieval systems, which often return a single chunk to answer a query. If that chunk lacks context, the model's answer may be vague or incorrect.

There are two common types of overlap:

- Fixed overlap
- Dynamic overlap

Fixed overlap consistently repeats a set number of tokens or sentences between chunks. For example, if you're using 100-token chunks and set a 20-token fixed overlap, each new chunk will begin 80 tokens after the previous one. This method is simple to implement and works well when the text structure is uniform and the transitions between chunks are predictable. It's commonly used in systems that process structured data, such as logs or transcripts, where the size and pacing of the content are consistent.

In technical terms, if your text is being divided into chunks of 100 tokens and you move forward by 80 tokens each time, there's a 20-token overlap between each new chunk and the one before it. This repeated portion helps preserve continuity so that important details aren't lost when one chunk ends and another begins.

This method is great when your data is consistent and evenly structured, such as the following:

- Log files (each entry follows the same pattern)
- Meeting transcripts (with clear, evenly spaced dialogue)
- Sensor data or structured reports

It's also easy to implement with code, which makes it a popular default strategy for beginner projects or systems that handle predictable data formats.

With *dynamic overlap*, instead of repeating a fixed number of tokens, the approach uses content-aware rules to decide where chunks should begin and end. For example, if a chunk is about to cut a sentence in half, dynamic overlap would automatically extend the chunk boundary to include the full sentence. Similarly, if a paragraph transitions into a related idea, the chunk might stretch slightly to preserve that flow.

Dynamic overlap is particularly useful when the content relies heavily on narrative structure, logical progression, or fluid explanation, such as in technical documentation, essays, or instructional materials. These types of documents often present concepts that span several sentences or paragraphs. Dynamic overlap ensures that the model always receives enough context to understand the full idea, even if the chunk starts in the middle of a longer discussion.

For example, think about reading a story: you wouldn't want to start reading in the middle of a sentence or stop right before the plot twist. Dynamic overlap avoids those problems by intelligently adjusting chunk boundaries.

The trade-off is that this method may produce chunks of varying sizes, which adds some complexity when indexing and retrieving them. However, the benefit is improved coherence and higher-quality retrieval results, particularly in semantically dense or conceptually layered texts.

Choosing between fixed and dynamic overlap depends on your data and goals. Fixed overlap is easier to manage and scale but may miss nuanced transitions. Dynamic overlap offers richer context but requires more sophisticated processing.

Example 3-2 walks through a practical example using a sliding window approach to implement overlap in Python.

Example 3-2. Sliding window overlap in Python

```
def sliding_window_chunking(text, window_size=100, stride=80):
    """
    Splits text into overlapping chunks using a sliding window approach.
    window_size: number of tokens per chunk
    stride: number of tokens to move the window each step (controls
overlap)
    """
    import tiktoken  # For tokenization (OpenAI-compatible)

    enc = tiktoken.get_encoding("cl100k_base")
    tokens = enc.encode(text)

    chunks = []
    for i in range(0, len(tokens), stride):
        chunk = tokens[i:i + window_size]
        if chunk:
            chunks.append(enc.decode(chunk))

    return chunks
```

In this example, a chunk contains 100 tokens and moves forward by 80 tokens each time, so there's a 20-token overlap. You can adjust the `stride` to control the frequency of repetition between chunks.

Impact of Chunking on Retrieval

Now that we've explored different chunking strategies and how to control chunk size and overlap, it's time to examine why chunking matters so much in the first place, especially in the context of RAG systems. The way you divide documents into chunks affects both the *retrieval accuracy* and the *generative quality* of responses produced by LLMs.

When you perform a retrieval query in a RAG pipeline, you typically retrieve one or more top-ranked chunks that are semantically similar to the user query. These chunks are then passed to the language model along with the prompt. If your chunks are too large, you might include unrelated information that reduces relevance. If they're too small, you might miss vital context needed to understand or answer the query.

The impact of chunking can be summarized in three areas:

Retrieval relevance

> Precise chunking increases the likelihood that retrieved documents will match the user's intent by aligning closely with the granularity of the user's question. For example, suppose a user asks about a specific step in a process. In that case, well-defined chunks that isolate that step (such as a heading or bullet list) are more likely to be retrieved than large sections that bury the information in unrelated details. Conversely, if the chunking is too coarse—such as paragraph-level for a multi-topic article—the retrieval system may return entire blocks that include extraneous or misleading content, weakening the downstream answer quality.

Response coherence

> The language model can only generate coherent responses if it receives enough contextual information. Inadequate chunking—such as splitting a definition across two chunks or isolating follow-up steps from their initial instruction—weakens the model's ability to synthesize and generate

logical, connected outputs. For instance, if an onboarding process is split such that *Step 1* and *Step 2* are stored in separate chunks, the model may generate an incomplete or incorrect response if only one step is retrieved. Maintaining semantic cohesion within a chunk ensures that the language model operates with a comprehensive understanding of the content, enabling it to deliver more natural and accurate responses.

Latency and performance

Finer-grained chunking increases the total number of chunks, which can significantly impact the system's latency and computational efficiency. Each chunk must be evaluated and compared against a user query during retrieval, and a larger number of chunks increases the search space. This results in slower response times and higher processing overhead, especially when working with large datasets. For instance, in a customer support scenario where hundreds of documents are split into small 50-token chunks, the retrieval engine must scan through thousands of pieces instead of hundreds, potentially increasing query latency and compute costs. Therefore, teams must weigh the benefits of high retrieval granularity against the constraints of infrastructure and performance.

These concepts may be seen more clearly through an illustrative example.

Real-World Example: Customer Support Knowledge Base

Imagine a company with a large knowledge base containing articles on troubleshooting software. If they chunk each article at the paragraph level (coarse chunking), a query like *How do I reset my password if I forgot my*

security question? may retrieve a long chunk that discusses account issues in general but buries the specific reset procedure midway through.

Now, consider a more refined chunking strategy that separates content based on subheadings or user-intent-aligned sections. One chunk might specifically cover “Password Recovery Options.” In this case, the retrieval system is far more likely to surface the right chunk, and the language model can generate an accurate, targeted response.

Visualizing the Trade-off

How chunking choices affect performance for multiple parameters. Table 3-2 illustrates the trade-off associated with selecting specific chunking options.

Table 3-2. How chunking choice affects performance.

Chunk Size	Retrieval Relevance	Context Coverage	System Latency
Small	High	Low	High
Medium	Balanced	Balanced	Moderate
Large	Low	High	Low

As the table shows, chunking is a balancing act. You want to find a chunk size and an overlap strategy that maintains context while avoiding noise or inefficiency.

> **TIP**
>
> Start with medium-sized chunks (150–300 tokens) and test retrieval performance using evaluation metrics such as precision, recall, or MRR (mean reciprocal rank). Adjust based on the specificity of your documents and user queries.

The next section examines how to clean and filter content before chunking, including handling PDFs, noisy input, and extracting data from unstructured formats.

Content Filtering and Extraction

Before you can begin chunking or building retrieval pipelines for a RAG system, it's essential to clean and prepare your raw content. The quality of your input data directly influences what the language model retrieves and generates. If your documents contain noise, redundancy, or irrelevant information, even the best chunking strategy won't prevent poor outcomes downstream.

This section covers the critical preprocessing techniques that ensure your source data is reliable, consistent, and semantically valuable. You'll learn how to identify and remove common types of noise—like duplicated text, irrelevant headers, or formatting artifacts—and how to handle unstructured formats such as scanned PDFs and image-based documents. You'll also explore how to convert semi-structured data into optimized, query-ready formats, such as Delta Lake tables, which enhance retrieval performance and scalability.

Content filtering is not just about removing what's unnecessary—it's about enhancing what matters. Whether your source is a messy document scan or a verbose product manual, applying effective preprocessing ensures that your RAG pipeline operates on high-signal content, leading to better semantic search and more accurate generative responses.

The following subsections break down this preparation process into three parts: eliminating noise, extracting content from complex formats, and preparing your cleaned data for scalable querying.

Removing Redundancy and Noise

Before diving into document chunking or semantic indexing, it's crucial to clean your data by removing redundant and noisy content. *Redundancy*

refers to unnecessary repetition of information, such as duplicated paragraphs, recurring boilerplate text, or templated headers. *Noise* includes artifacts that don't contribute to the semantic value of your document, like page numbers, HTML tags, timestamps, watermarks, or navigation menus.

In a RAG pipeline, such noisy elements can negatively affect both the retrieval and generation stages. For instance, if a chunk includes a repeated disclaimer at the bottom of every page, the retrieval engine may rank that chunk higher than more contextually relevant content, simply due to term frequency. Similarly, if boilerplate headers are not filtered out, the model may generate irrelevant or misleading completions.

Common sources of redundancy and noise include the following:

- Web crawled documents with footers, cookie notices, and ads
- PDFs that include repeated headers and footnotes on each page
- Corporate policy documents that embed disclaimers and legal text in every section
- OCR-scanned documents with garbled characters or markup tags

Real-World Example: Software Documentation Portal

Imagine a software company indexing its entire help center for a RAG chatbot. If each article contains repeated elements, such as a navigation menu, search bar instructions, or "last updated" timestamps, these noisy parts will appear in nearly every chunk. When a user asks, *How do I connect to the API?*, the retrieval engine might return chunks containing these common phrases rather than the actual API instructions, degrading the model's relevance.

To avoid this, you must implement a preprocessing step that scans for and removes repetitive elements using pattern matching, content heuristics, or visual structure detection.

Example 3-3 uses regex to remove noisy patterns.

Example 3-3. Removing common noisy patterns using regex

```
import re
def clean_text(text):
    # Remove HTML tags
    text = re.sub(r'<[^>]+>', '', text)
    # Remove repeated disclaimers or footers
    text = re.sub(r'This document is confidential.*?\n', '', text,
flags=re.IGNORECASE)
    # Remove timestamps and page numbers
    text = re.sub(r'Page \d+ of \d+', '', text)
    text = re.sub(r'Last updated: .*?\n', '', text)
    return text
```

> **TIP**
>
> Always inspect your documents manually before defining noise-removal rules. Noisy content often follows implicit patterns that differ across formats and organizations.

Extracting Content from PDFs and Images

You can extract meaningful text from PDFs and images using optical character recognition (OCR) and document parsing libraries. Many enterprise documents are stored as PDFs or scanned image files. These formats often contain valuable information, but their structure can make automated data extraction difficult. Before such content can be chunked or indexed in a RAG pipeline, it must first be converted into clean, readable text.

PDFs are challenging because they often include multiple columns, embedded images, irregular layouts, and artifacts like footnotes or page numbers. Scanned image-based PDFs pose an additional hurdle, as they require OCR to convert visual content into machine-readable text.

Depending on the document type, you might use a mix of tools:

- PyMuPDF or pdfminer.six for extracting text from digital PDFs with standard formatting.

- Tesseract OCR (via Python libraries like pytesseract) for scanned or image-based documents.
- Adobe PDF Services API for more structured output, particularly from complex or formatted PDFs.

Example 3-4 uses the PyPDF2 library to extract the Raw data from a PDF file.

Example 3-4. Extracting raw text from a PDF using PyPDF2

```
from PyPDF2 import PdfReader
reader = PdfReader("sample.pdf")
all_text = ""
for page in reader.pages:
    all_text += page.extract_text()
```

If working with image-based files, OCR is essential. Example 3-5 uses Tesseract OCR:

Example 3-5. Extracting text from scanned images using Tesseract OCR

```
import pytesseract
from PIL import Image
image = Image.open("scanned_doc.png")
text = pytesseract.image_to_string(image)
```

> **WARNING**
>
> OCR results can be noisy or inaccurate, especially if the scan quality is low. Always verify and clean extracted content before indexing.

An insurance company might digitize claim documents submitted as scanned forms and PDFs. To build a semantic search system that helps agents retrieve prior claims by condition or location, the documents must first be parsed using OCR. The extracted content is then cleaned and chunked, allowing accurate retrieval based on medical conditions, policy IDs, or reported damages.

Extracting content from PDFs and images is a prerequisite step in many real-world RAG pipelines. Once complete, the cleaned data can be transformed into Delta tables, enabling structured querying, explored next.

Converting to Delta Format for Querying

Once your source documents have been cleaned and transformed into readable text, the next step is to structure that data in a format optimized for large-scale, performant querying. Delta Lake is the preferred storage layer for this purpose in most Databricks-based RAG systems. It brings ACID transaction guarantees, scalable metadata handling, and native support for versioning—making it ideal for indexing chunked documents.

Unstructured or semi-structured data becomes much easier to retrieve and manage when stored in Delta format. After chunking, each document segment can be saved as a row in a Delta table, complete with metadata such as source document ID, chunk index, embedding vector (if applicable), and timestamp. This format facilitates efficient retrieval via SQL queries or vector similarity searches.

Why use Delta Lake in RAG pipelines?

There are several compelling reasons to use Delta Lake for RAG pipelines:

Query efficiency

> Delta Lake supports highly optimized reads through several advanced mechanisms. *Predicate pushdown* enables the system to skip scanning unnecessary data by applying filters early during query execution. *Indexing* ensures faster row access when querying specific metadata, such as chunk identifiers or document IDs. *Caching* enables frequently accessed chunks to remain in memory, significantly reducing latency in real-time search applications. These features collectively enable scalable, low-latency querying across large datasets—a crucial requirement for responsive RAG pipelines.

Incremental updates

Delta Lake enables you to update specific document chunks or add new ones without requiring a complete rewrite of the entire dataset. This is especially important in RAG pipelines where data sources are continuously evolving. For instance, if a new policy document is added or a section of an existing document is revised, only the affected chunks need to be updated. This makes the pipeline more efficient and scalable, reducing processing time and storage overhead. Additionally, Delta Lake supports merge operations (upserts), which help ensure that updates are applied accurately and efficiently.

Auditability and version control

Delta Lake maintains a transaction log that records every change made to a table, including data inserts, updates, and deletes. This provides a complete audit trail, which is particularly important in regulated industries such as finance, healthcare, or legal services. Additionally, Delta supports *time travel*—a feature that allows you to query data as it existed at a previous point in time. This is helpful during debugging, performance benchmarking, and model evaluation, as it enables you to compare results across different dataset versions or restore prior states after incorrect updates.

Compatibility

Delta format integrates seamlessly with various components of the Databricks ecosystem, making it highly versatile for machine learning workflows. For example, it connects with MLflow to track experiments and register models alongside the data used to train them, ensuring reproducibility. It works with the Databricks Feature Store to share and reuse

features across projects. It also supports popular vector search libraries, such as FAISS and Databricks-native vector search, enabling high-performance semantic retrieval. This tight integration across the stack accelerates development and streamlines operationalization of RAG systems.

Example conversion workflow

Example 3-6 is a simplified pipeline to convert extracted text chunks into a Delta table.

Example 3-6. Writing chunked documents to Delta Lake

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import monotonically_increasing_id
spark = SparkSession.builder.getOrCreate()
# Sample list of text chunks
chunks = ["Chunk 1: How to configure the system...", "Chunk 2:
Troubleshooting common errors..."]
# Convert to Spark DataFrame
df = spark.createDataFrame([(i, chunk) for i, chunk in
enumerate(chunks)], ["chunk_id", "content"])
# Save as Delta Table
df.write.format("delta").mode("overwrite").save("/mnt/delta/rag_chunks")
```

> **TIP**
>
> Always include metadata fields like document title, chunk number, and source URI when creating your Delta table. This makes downstream debugging and traceability easier.

Real-world example: Legal document indexing

A law firm digitizes court case transcripts and legal memos. After preprocessing and chunking, each chunk is stored in a Delta table with additional metadata like case ID, date, jurisdiction, and legal category. This enables legal researchers to run precise SQL-based searches and semantic queries across thousands of cases without having to reprocess raw files.

Storing your cleaned and structured content in Delta format is the final step in preparing high-quality input data for RAG applications. With chunked text in Delta tables, you're ready to build your retrieval layer and evaluate its performance, explored in the next section.

Evaluating Retrieval Quality

Once you've prepared your chunked data and structured it in a format like Delta Lake, the next critical phase is evaluating how well your system retrieves relevant information in response to user queries. High-quality retrieval is essential in any RAG pipeline because the language model's output is only as good as the context it receives.

This section focuses on the performance of the retrieval layer—how well your system surfaces the right chunks at the right time. Poor retrieval leads to irrelevant or misleading generations, which can severely undermine user trust and model utility. To ensure your RAG system consistently delivers valuable, accurate responses, you need to evaluate the retrieval pipeline using robust metrics and diagnostic methods.

We'll begin by examining foundational retrieval metrics such as precision and recall, and how reranking strategies can further enhance relevance. Then we'll look at how to interpret those metrics to improve system performance. Finally, we'll explore common issues like retrieval gaps and how to diagnose and resolve them.

By the end of this section, you'll be equipped with the skills to apply evaluation metrics such as precision and recall to assess how well your retrieval system is performing. You'll understand how to fine-tune chunking and indexing parameters to improve the relevance of retrieved content, and you'll be able to diagnose and resolve retrieval gaps using structured feedback and diagnostic techniques.

Let's begin with an overview of core retrieval metrics and their implications in the RAG pipeline.

Precision and recall

Understanding how to assess the effectiveness of your retrieval system is essential for building reliable RAG pipelines. Precision and recall are foundational metrics borrowed from information retrieval that help quantify how relevant and comprehensive your retrieved results are. These metrics offer a data-driven way to evaluate whether your retrieval system is surfacing the right chunks to feed into the language model.

Precision

> Precision measures the proportion of retrieved chunks that are actually relevant to the query. In simpler terms, it helps you understand how many of the documents or passages your system retrieved are truly useful for answering the question. For example, if your RAG system retrieves 10 chunks in response to a user query and 7 of them are directly related to the question, the precision is 70%. High precision ensures that the language model operates with accurate and trustworthy context, which is especially important in high-stakes domains such as finance or healthcare.

```
Precision = (Number of Relevant Chunks Retrieved) / (Total Chunks
Retrieved)
```

Recall

> Recall measures the proportion of all relevant chunks that were successfully retrieved. It answers the question: Out of everything that should have been retrieved, how much did we actually get? For example, if a query has 20 truly relevant chunks in your document store and your system retrieves 12 of them, then the recall is 60%. High recall is especially critical in scenarios where missing information can lead to incomplete or biased responses, such as academic research or regulatory compliance. A system

> optimized for high recall ensures that as much relevant content as possible is surfaced, even if some irrelevant chunks are included.

```
Recall = (Number of Relevant Chunks Retrieved) / (Total Relevant
Chunks Available)
```

High precision means the retrieved content is accurate, whereas high recall ensures that you're not missing important context. In practice, there's often a trade-off—retrieving more documents may increase recall but reduce precision.

Example scenario: Imagine you're building a legal RAG assistant that retrieves past court cases related to a specific legal issue. If the system retrieves 10 documents, and 7 of them are relevant to the query while 3 are not, your precision is 70%. If 10 other pertinent documents of the corpus weren't retrieved, your recall would be 41% (7 retrieved out of 17 total relevant).

Reranking: Improving relevance

Initial retrieval is often based on vector similarity using cosine distance or dot product, but this isn't always enough. *Reranking* is a post-processing step where the top-N results from a vector search are re-evaluated using a more sophisticated model—often a cross-encoder or a fine-tuned BERT variant—that takes into account both the query and the full document content.

This approach improves ranking quality by modeling semantic alignment more accurately. Reranking is particularly useful when handling long-form queries or complex topics that necessitate a thorough understanding. Example 3-7 uses a cross-encoder to rerank retrieved chunks.

Example 3-7. Using a cross-encoder for reranking retrieved chunks

```
from transformers import pipeline
reranker = pipeline("text-classification", model="cross-encoder/ms-
marco-MiniLM-L-6-v2")
query = "What are the legal implications of data privacy violations?"
candidate_chunks = ["Chunk A: The court ruled in favor...", "Chunk B:
Data privacy laws require...", ...]
scored_chunks = [(chunk, reranker(f"{query} [SEP] {chunk}")[0]
['score']) for chunk in candidate_chunks]
sorted_chunks = sorted(scored_chunks, key=lambda x: x[1],
reverse=True)
```

TIP

Reranking improves precision significantly but can introduce latency. Use it selectively on a small candidate set (for example, the top 100 chunks).

When to use precision, recall, and reranking

Precision is best applied in scenarios where accuracy is paramount and retrieving incorrect information could have serious consequences, such as in healthcare diagnosis tools or legal document retrieval systems. In these contexts, surfacing only highly relevant content helps prevent the spread of misinformation or misinterpretation.

Recall is more critical in situations where missing out on relevant information is riskier than including some irrelevant data. For example, in scientific research or legal discovery, it’s important to retrieve as many potentially useful documents as possible to ensure thorough coverage of the topic.

Reranking should be used when your initial retrieval method brings back loosely relevant results that need refinement. This is especially useful in cases involving complex, multi-part queries or when user intent is subtle—like searching customer support records for nuanced user sentiment or looking up context-specific enterprise knowledge.

Together, these methods form the foundation for evaluating and enhancing the retrieval layer in your RAG pipeline.

Using Metrics to Improve Performance

Evaluating retrieval quality is not just about reporting numbers—it's about identifying opportunities to tune and optimize your system. This section explores how to use precision, recall, and other metrics as actionable levers for performance improvement in RAG pipelines. We'll also examine how to identify bottlenecks, validate chunking strategies, and introduce feedback loops that continuously refine retrieval quality.

From Metrics to Action

Precision and recall are useful starting points, but interpreting these metrics in isolation isn't enough. The goal is to link them to specific areas of your pipeline that can be improved. For example, low precision may indicate noisy chunks or overly broad chunking strategies, while low recall could suggest gaps in document coverage or poor vector indexing.

Let's say your RAG system shows 60% precision and 30% recall. Rather than labeling these values as simply *good* or *bad*, it's important to treat them as indicators pointing to deeper systemic issues. For example:

- If the chunks are too short, they might not contain enough context for the model to make accurate predictions. If they're too long, key information could be diluted or buried, causing retrieval mismatches.
- Documents might still contain irrelevant content like boilerplate legal text, navigation menus, or repetitive footers. This adds noise and reduces precision.
- Query embeddings may not accurately capture user intent if the embedding model isn't trained on domain-specific language. Alternatively, if the corpus was indexed using a different embedding model or encoding logic, semantic mismatches can occur.

These upstream factors create bottlenecks in retrieval quality, and identifying them allows you to test corrective strategies such as tuning chunk size, introducing preprocessing filters, or aligning your embedding models more closely with the domain vocabulary.

You can adjust chunk size, tune the similarity threshold, or experiment with different embedding models to boost both precision and recall over time. Table 3-3 shows the impact of chunk size on precision and recall.

Table 3-3. Impact of chunk size on precision and recall

Chunk Size	Precision	Recall	Comments
50 tokens	0.85	0.45	High precision, low recall
150 tokens	0.75	0.65	Balanced trade-off
300 tokens	0.55	0.80	High recall, lower precision

> **WARNING**
>
> Avoid over-optimizing a single metric—raising precision too much might drop recall and vice versa. Always strike a balance based on your RAG use case.

Validating Chunking and Filtering Strategies

Use metric-based feedback to assess the effectiveness of your chunking and content filtering strategies. For instance, test different chunk sizes and overlaps and compare their impact on retrieval quality using a validation set.

Example 3-7 evaluates retrieval performance using precision and recall. Suppose you have a list of relevant document IDs for a query and a separate

list of document IDs retrieved by your RAG system. You want to see how many of the retrieved results are correct.

Example 3-7. Computing retrieval precision and recall from a validation set

```
from sklearn.metrics import precision_score, recall_score

# List of relevant document IDs (e.g., gold standard)
relevant_docs = {"doc1", "doc3", "doc5", "doc6"}

# Retrieved document IDs for a sample query
retrieved_docs = ["doc1", "doc2", "doc4", "doc5"]

# Create binary relevance vectors for evaluation
true_labels = [1 if doc in relevant_docs else 0 for doc in
retrieved_docs]
predicted_labels = [1] * len(retrieved_docs)  # All retrieved docs are
considered predicted relevant

# Calculate precision and recall
precision = precision_score(true_labels, predicted_labels)
recall = len(set(retrieved_docs) & relevant_docs) / len(relevant_docs)

print("Precision:", round(precision, 2))
print("Recall:", round(recall, 2))
```

In this example, precision measures the proportion of retrieved documents that are relevant, and recall measures the proportion of appropriate documents that your system was able to recover.

> **TIP**
>
> Use hold-out datasets with known relevance labels to run retrieval evaluation offline. This avoids relying solely on user interaction metrics.

Monitoring and Feedback Loops

To sustain high performance in production, establish a monitoring loop. Track retrieval metrics over time, ideally broken down by query category, document type, or user segment. You can also introduce human-in-the-loop

feedback where users label retrieved chunks as useful or not, helping refine both chunking logic and embedding models.

For example, a customer support bot could allow agents to mark helpful context chunks. These markings can then inform the retraining of the chunking strategy or reranking model, gradually improving both relevance and completeness.

Together, metrics aren't just passive measurements—they become diagnostic and optimization tools that drive better outcomes in the RAG system.

Troubleshooting Retrieval Gaps

Even the best-designed RAG systems will encounter retrieval gaps—moments when relevant documents are not returned, or the retrieved chunks lack sufficient context for generating meaningful responses. These gaps can significantly degrade model output, reduce trust in the system, and render downstream applications, such as summarization or Q&A, unreliable.

This section equips you with strategies to identify and resolve such retrieval blind spots. You'll learn how to debug missing or poor results, use logs and metrics to understand failure patterns, and apply techniques to improve recall and context coverage. This is especially important in regulated or high-stakes industries, such as healthcare or finance, where incomplete retrievals can lead to inaccurate or even harmful results.

Troubleshooting retrieval gaps is not just about fixing individual cases; it's about improving the systemic reliability of your pipeline. This section focuses on how to proactively detect weak points and implement solutions such as refining chunking strategies, enhancing indexing quality, introducing fallback query mechanisms, and combining retrieval methods to boost robustness.

Let's begin by examining the primary causes of retrieval failure and outlining practical methods for diagnosing them in a RAG pipeline.

Common Causes of Retrieval Gaps

Retrieval gaps typically arise when the system is unable to return relevant or complete results for a given user query. These gaps often manifest as missing, partial, or semantically unrelated chunks, and can significantly impact the performance of your RAG pipeline. Understanding the root causes of such failures is crucial to ensure reliable document retrieval, particularly in high-accuracy applications such as legal research, medical support tools, or compliance systems. The primary causes of retrieval gaps can be broadly categorized into three areas.

Ineffective chunking

If your chunking strategy splits logical units of information across chunks —or uses sizes that are too small or too large—it can cause context loss or dilution. Granularity refers to the level of detail or size of each individual chunk in your document corpus. If the granularity is too fine (for example., small phrases or short sentences), the resulting chunks may not contain enough contextual information to be relevant on their own. On the other hand, overly coarse chunks (large sections or multiple paragraphs) may include too much unrelated content, increasing noise and reducing retrieval precision. For instance, slicing a technical specification mid-sentence can result in fragments that hold little semantic value, making them less likely to be retrieved when needed. Conversely, a large chunk covering multiple features may dilute relevance when only one specific term is queried. Choosing the right chunk size depends on the nature of your documents and the specificity of the user queries.

Embedding mismatch

When the query embedding and document embedding come from incompatible models or domains, even semantically similar content may not align. A general-purpose embedding model is typically trained on a broad range of internet text, such as Wikipedia or Common Crawl, and aims to perform well across a wide variety of topics. However, it may not capture specialized vocabulary, stylistic nuances, or domain-specific semantics

required for fields like law, finance, or medicine. A domain-specific query—for example, *interpretation of GAAP principles in earnings restatement cases*—contains technical terminology and contextual expectations that general-purpose models may fail to encode properly. As a result, when such a query is used against financial filings or regulatory documents, the cosine similarity between the query and relevant chunks may be low, leading to poor retrieval performance.

Indexing errors or gaps

If your document preprocessing or indexing pipeline skips important data—such as figures, tables, or appendices—your retrieval layer will lack access to this information, leading to incomplete or misleading results. This issue frequently occurs when dealing with complex documents like PDFs, especially those containing multi-column layouts, embedded images, or footnotes. In such cases, basic text extractors may misinterpret the document structure, resulting in missing or jumbled text. For example, a financial report that includes tabular breakdowns of quarterly earnings may lose critical context if the parser fails to capture the tables correctly. This can severely impact RAG accuracy when users issue queries expecting numerical or tabular data.

> **TIP**
>
> To detect whether embedding mismatch is the issue, try running manual similarity checks between your query and a few expected relevant chunks. If the similarity score is low, it's a sign you may need to align your embedding models or fine-tune them for your specific corpus.

Corrective Techniques to Resolve Retrieval Gaps

Once a retrieval gap is identified—whether due to chunking, embeddings, or indexing—several targeted techniques can be employed to improve overall retrieval effectiveness. The goal is not just to patch isolated failures but to enhance the robustness and adaptability of your RAG pipeline. By

systematically applying corrective measures, you ensure that the system retrieves high-quality, contextually relevant information across a variety of scenarios and query types:

Revisiting chunking strategy

If the root cause is poor chunking, consider switching to a strategy that preserves semantic boundaries. This means designing chunking logic that respects natural divisions in content—such as paragraphs, sections, or topics—rather than cutting content at arbitrary token or character counts. For example, switching from fixed-length chunking to paragraph-based or semantic chunking allows the model to retrieve more coherent and meaningful content during generation. Semantic chunking, in particular, relies on NLP techniques or model-generated embeddings to group sentences with high contextual affinity. Additionally, introducing controlled chunk overlap (where a portion of one chunk is repeated in the next) ensures that important transitional phrases or boundary content aren’t lost between adjacent segments. This is especially valuable for documents where critical meaning spans across paragraphs, such as legal arguments or multi-step procedures in technical manuals.

Aligning embedding models

Ensure that both the query and document encoders come from the same model family or are fine-tuned on similar data. If the query encoder is trained on general-purpose internet text but the documents are specialized—say, financial statements or medical transcripts—the embedding vectors may lie in different semantic spaces, reducing their alignment. For domain-specific use cases, consider training sentence transformers using contrastive learning on labeled pairs, such as matching medical questions with diagnostic explanations or legal inquiries with corresponding statutes. This approach helps create embeddings that better reflect both the structure and vocabulary of your documents, leading to improved retrieval accuracy.

Enhancing the indexing pipeline

Review your document parsing and ingestion process thoroughly to ensure that no relevant information is lost during the preprocessing stage. Basic text extractors may fail to capture complex layouts, embedded tables, or figures, particularly when working with scanned PDFs or image-based documents. Instead, leverage specialized tools like Azure Form Recognizer, Adobe PDF Extract API, or Amazon Textract, which can parse structured content and extract text along with layout, tables, and key-value pairs. For example, in financial reporting or clinical trial documents, tables often contain critical data that cannot be accurately interpreted without proper formatting. To preserve this context, include metadata fields (e.g., document title, author, publication date) and structure content into labeled sections or fields wherever applicable. This not only aids retrieval but also enhances downstream semantic interpretation by the RAG model.

Query expansion and rewriting

When a user's query retrieves few or no relevant results, several techniques can help improve recall. One approach is to apply synonym expansion, where alternative expressions or terminology are automatically added to the query. For example, expanding *heart attack* to include *myocardial infarction* can help match more technical content in medical records. Spelling normalization—especially important in OCR-scanned documents or user-entered inputs—can correct typos or variations in spelling (*color* vs. *colour*). Another powerful strategy is to rephrase the query using an LLM to generate semantically equivalent variants. This technique enables the system to reformulate user intent in a manner that better aligns with document phrasing. If dense vector search still returns low scores, fallback mechanisms like keyword-based search using BM25 or TF-IDF can be activated. These sparse methods rely on exact term matching, which often succeeds when semantic search fails due to embedding misalignment or narrow vocabulary. Integrating these methods ensures broader query coverage and helps recover documents that would otherwise be missed.

Hybrid search techniques

Combine dense vector search with sparse retrieval methods to improve both recall and coverage in your RAG system. Start by using a sparse retriever like BM25 to fetch a wide range of potentially relevant documents based on exact keyword matches. This initial set serves as a candidate pool. Then, use dense retrieval techniques—such as computing cosine similarity between embeddings—to rerank these candidates based on their semantic relevance. Optionally, you can apply a cross-encoder model that takes the query and each candidate chunk as input to score relevance more precisely. This hybrid approach ensures that even if dense models miss a document due to embedding limitations, sparse retrieval provides a safety net to catch relevant results based on lexical matches.

> **NOTE**
>
> While hybrid search can significantly improve coverage, it may also increase latency. Always benchmark retrieval time against your application's performance constraints.

Use case: Legal document search engine

A legal tech firm implemented an RAG-based assistant to help lawyers retrieve case precedents based on natural language queries. Initially, the system returned limited and often irrelevant results when users searched for nuanced legal arguments. Investigation revealed two issues:

- Legal briefs were chunked using a fixed-length strategy that often split logical clauses across chunk boundaries.
- The embedding model used for indexing was not trained on legal language, resulting in semantic mismatches.

To fix this, the team reprocessed the corpus using paragraph-based chunking with overlap and replaced the embedding model with a domain-adapted version fine-tuned on legal Q&A pairs. They also layered a BM25-based fallback retrieval to catch keyword matches missed by the dense retriever. Following implementation, query coverage increased by 40%, and

lawyers reported a significant improvement in the relevance of the retrieved documents.

This example underscores the importance of aligning preprocessing and retrieval techniques with domain-specific content and use patterns.

Key takeaways

We have explored retrieval gaps and how to resolve them. Here are the key takeaways from our learning:

- Retrieval gaps can stem from ineffective chunking, mismatches in embedding, or indexing omissions.
- You can diagnose these issues by inspecting query-embedding similarity scores, analyzing retrieval logs, and testing query reformulations.
- Applying overlapping chunking, aligning embeddings, enhancing indexing, and hybrid search are effective remediation strategies.
- Domain-specific use cases require tailored adjustments to maximize retrieval precision and recall.

Table 3-3 compares some retrieval gap strategies.

Table 3-4. 3 Diagnostic and Resolution Flow for Retrieval Gaps

Step	Diagnostic Question	Action to Take
1	Are chunks too small or fragmented?	Switch to paragraph/semantic chunking
2	Are similarity scores unexpectedly low?	Align or fine-tune embedding models
3	Are certain document sections never retrieved?	Check the indexing pipeline for omissions
4	Do fallback queries retrieve relevant docs?	Add hybrid search to the production workflow

Now that you've explored methods for diagnosing and correcting retrieval gaps, the next step is to solidify your understanding with a practical assessment.

Practice Multiple-Choice Questions

Following are multiple-choice questions to reinforce your understanding of the concepts in this chapter. These are structured to reflect the style and expectations of the Databricks Generative AI Associate certification exam. The answer key appears at the end of the questions.

1. 1. What is the primary reason to apply overlapping in chunking strategies for Retrieval-Augmented Generation (RAG)?

    a. A. To reduce storage costs

    b. B. To ensure chunks contain complete and continuous context

c. C. To prevent duplication of documents

d. D. To reduce the number of chunks required for indexing

2. 2. Which of the following is a drawback of using fixed-length chunking?

    a. A. It is computationally expensive

    b. B. It preserves semantic coherence across all domains

    c. C. It can break logical units of meaning, reducing retrieval accuracy

    d. D. It requires a specialized model to implement

3. 3. What is the main goal of filtering redundant or noisy content before indexing?

    a. A. To improve document visualization

    b. B. To reduce the number of vector embeddings

    c. C. To enhance the relevance and precision of retrieval results

    d. D. To increase the storage space utilization

4. 4. In which scenario would semantic chunking be preferred over sentence-based chunking?

    a. A. When processing well-structured CSV files

    b. B. When dealing with loosely organized narratives or FAQs

    c. C. When minimizing token usage is the main objective

    d. D. When using a rule-based keyword retriever

5. 5. Which technique would most likely resolve embedding mismatch issues in domain-specific RAG pipelines?

a. A. Switching to fixed-length chunking

b. B. Lowering chunk size granularity

c. C. Using domain-adapted or fine-tuned embedding models

d. D. Removing stop words from queries

6. 6. Which of the following best describes hybrid retrieval in RAG systems?

    a. A. Indexing content using only BM25-based methods

    b. B. Using both sparse and dense retrieval techniques together

    c. C. Removing fallback mechanisms to speed up search

    d. D. Separating queries by topic before performing retrieval

Table 3-5. Answer Key:

Question	Answer	Explanation
1	B	Overlapping ensures that important context isn’t lost across chunk boundaries.
2	C	Fixed-length chunking may split logical units, affecting semantic coherence.
3	C	Removing redundant or noisy content helps improve the quality of retrieved results.
4	B	Semantic chunking is ideal for free-form narratives where meaning spans multiple sentences.
5	C	Domain-specific embeddings align better with specialized queries and content.
6	B	Hybrid retrieval combines the strengths of sparse (keyword) and dense (semantic) methods.

Hands-On Lab: Chunking and Indexing for Retrieval-Augmented Generation (RAG)

Scenario

You are a data engineer working for a health research organization. Your team is building a Retrieval-Augmented Generation (RAG) system to help researchers, doctors, and analysts query critical documents like CDC guidelines, WHO protocols, and medical articles using large language models (LLMs).

To ensure the system returns highly relevant and accurate results, you must:

- Clean and preprocess semi-structured documents,
- Apply appropriate chunking strategies,
- Generate embeddings,
- Store the results in Delta Lake,
- And finally, index them for semantic search using Databricks Vector Search.

Your goal in this lab is to simulate a real-world pipeline and observe how chunk size and granularity impact the quality of retrieved answers.

Objective

By the end of this lab, you will be able to:

- Extract and clean text from public PDF medical documents
- Apply sentence-based chunking with overlap to create coherent chunks
- Convert chunked text into Delta format and generate vector embeddings using a hosted embedding endpoint
- Index the embedded chunks using Databricks Vector Search
- Perform similarity-based queries using natural language prompts
- Simulate how chunk size affects retrieval precision using a post-query evaluation

NOTE

You can access this lab on Github at link.

About the Author

Rajaniesh Kaushikk is a seasoned technology expert with over 20 years of experience in complex software application architectures for Fortune 500 companies. As a Microsoft MVP, Databricks MVP, and TOGAF Certified Enterprise Architect, he specializes in data integration, governance, analysis, and advanced technical architecture. Rajaniesh is also a Microsoft Certified Azure Solutions Architect Expert and a Databricks Champion, known for his proficiency in Azure Cloud and various data platforms. He is a frequent speaker at Microsoft technology events and has a passion for educating and mentoring technology professionals.